

Aum Sri Sairam

Perceptron and Perceptron Algorithm

Questions

- What is a perceptron?
- What type of task performed by PA and which decision boundary is used for the task ?
- How the task achieved by the PA?
- What is the geometric interpretation of PA?
- Write and explain the PA.

Task and Boundary

- Classification Task
- Hyperplane

How?

The *perceptron learning algorithm* tries to find a separating hyperplane by minimizing the distance of misclassified points to the decision boundary. If

If a response $y_i=1$ is misclassified then $h(x) < 0$

If a response $y_i = -1$ is misclassified then $h(x) > 0$

Geometric Intuition: SVM

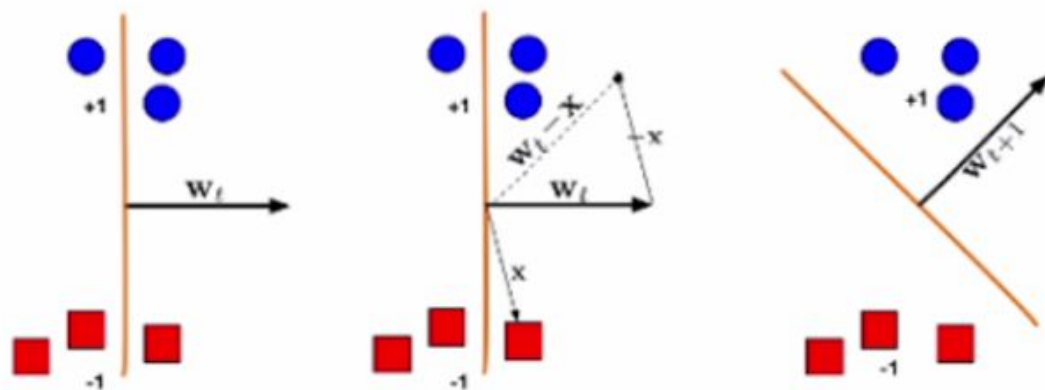


Illustration of a Perceptron update. (Left:) The hyperplane defined by w_t misclassifies one red (-1) and one blue (+1) point. (Middle:) The red point x is chosen and used for an update. Because its label is -1 we need to **subtract** x from w_t . (Right:) The updated hyperplane $w_{t+1} = w_t - x$ separates the two classes and the Perceptron algorithm has converged.

Algorithm:

Perceptron Algorithm

```
Initialize  $\vec{w} = \vec{0}$ 
while TRUE do
   $m = 0$ 
  for  $(x_i, y_i) \in D$  do
    if  $y_i(\vec{w}^T \cdot \vec{x}_i) \leq 0$  then
       $\vec{w} \leftarrow \vec{w} + y_i \vec{x}_i$ 
       $m \leftarrow m + 1$ 
    end if
  end for
  if  $m = 0$  then
    break
  end if
end while
```

```
// Initialize  $\vec{w}$ .  $\vec{w} = \vec{0}$  misclassifies everything.
// Keep looping
// Count the number of misclassifications,  $m$ 
// Loop over each (data, label) pair in the dataset,  $D$ 
// If the pair  $(\vec{x}_i, y_i)$  is misclassified
// Update the weight vector  $\vec{w}$ 
// Counter the number of misclassification

// If the most recent  $\vec{w}$  gave 0 misclassifications
// Break out of the while-loop

// Otherwise, keep looping!
```

Artificial Neural Networks

Artificial neural networks or simply *neural networks* are inspired by biological neuronal networks.

A real biological neuron, or a *nerve cell*, comprises dendrites, a cell body, and an axon that leads to synaptic terminals. A neuron transmits information via electrochemical signals.

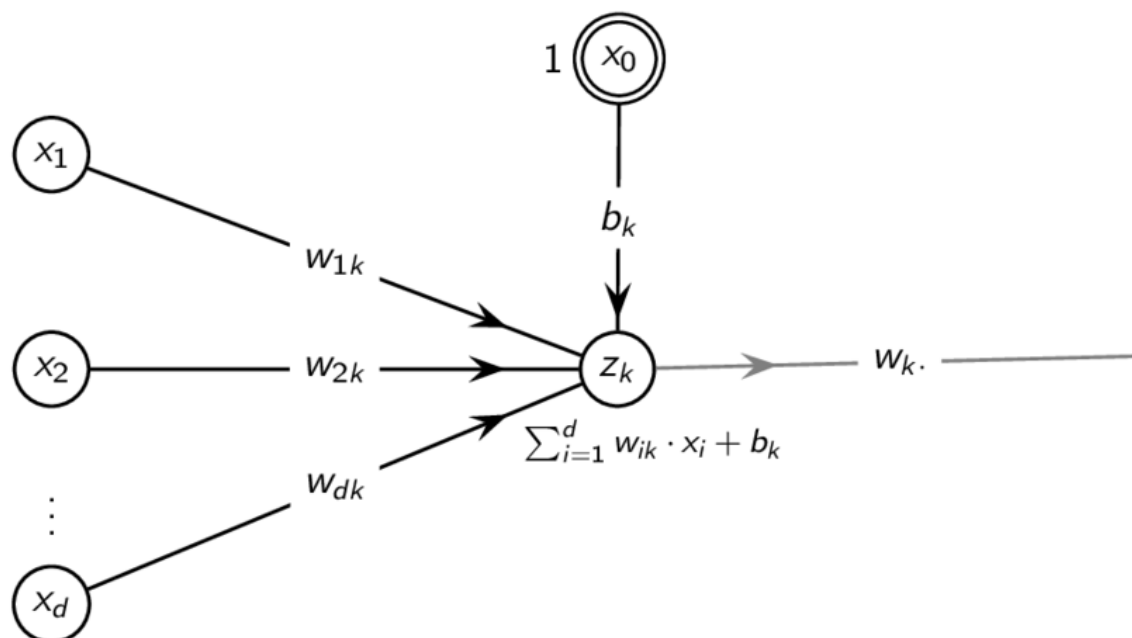
When there is enough concentration of ions at the dendrites of a neuron it generates an electric pulse along its axon called an *action potential*, which in turn activates the synaptic terminals, releasing more ions and thus causing the information to flow to dendrites of other neurons.

A human brain has on the order of 100 billion neurons, with each neuron having between 1,000 to 10,000 connections to other neurons.

Artificial Neural Networks

Artificial neural networks are comprised of abstract neurons that try to mimic real neurons at a very high level.

They can be described via a weighted directed graph $G = (V, E)$, with each node $v_i \in V$ representing a neuron, and each directed edge $(v_i, v_j) \in E$ representing a synaptic to dendritic connection from v_i to v_j . The weight of the edge w_{ij} denotes the synaptic strength.



Artificial Neuron

An artificial neuron acts as a processing unit, that first aggregates the incoming signals via a weighted sum, and then applies some function to generate an output.

A binary neuron outputs a 1 whenever the combined signal exceeds a threshold, or 0 otherwise.

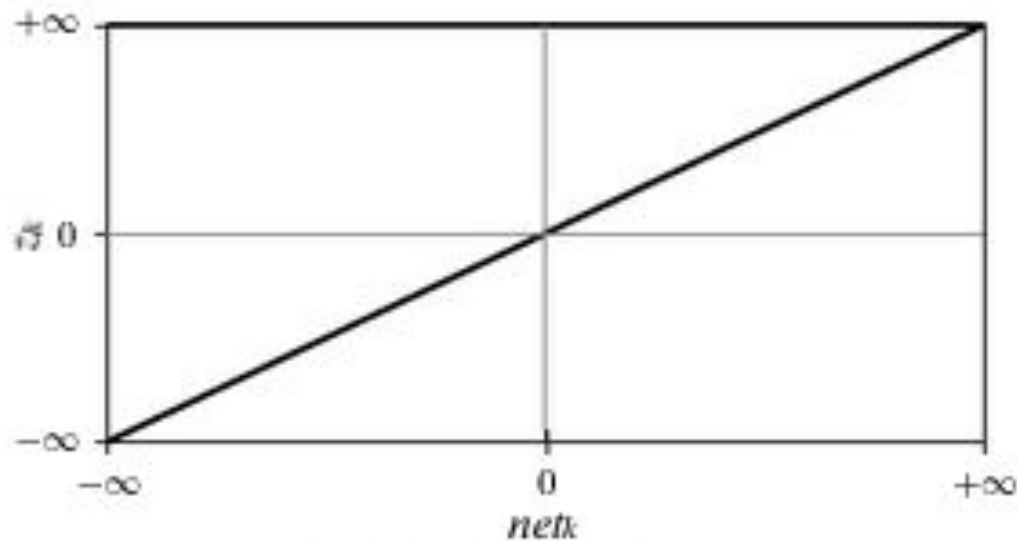
$$net_k = b_k + \sum_{i=1}^d w_{ik} \cdot x_i = b_k + \mathbf{w}^T \mathbf{x}$$

x_0 is a special *bias neuron* whose value is always fixed at 1, and the weight from x_0 to z_k is b_k .

Finally, the output value of z_k is given as some *activation function*, $f(\cdot)$, applied to the net input at z_k

$$z_k = f(net_k)$$

Linear Function:



(a) Linear (z_k versus net_k)

Linear/Identity Function: Identity is the simplest activation function; it simply returns its argument, and is also called a *linear* activation function:

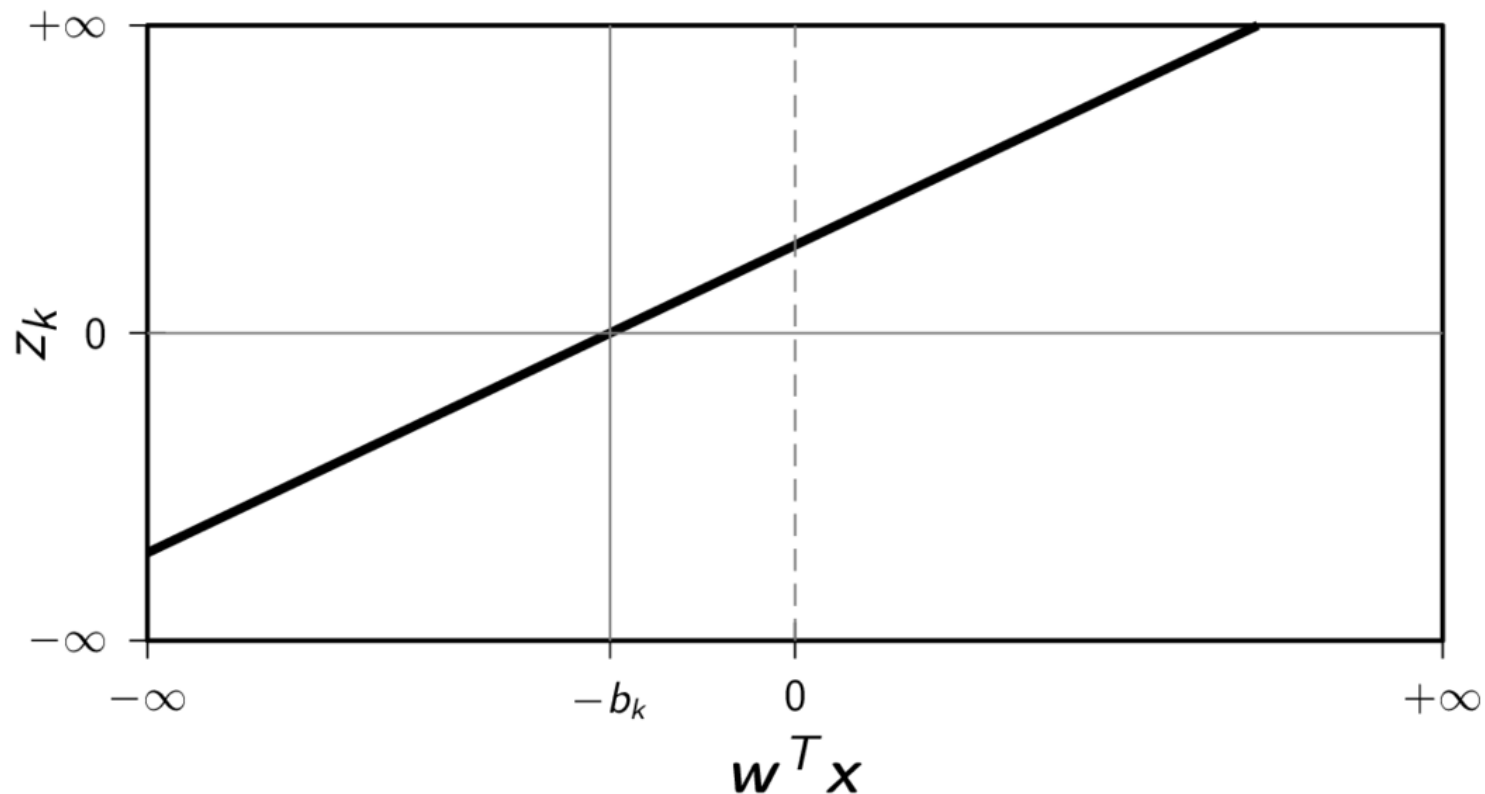
$$f(net_k) = net_k \quad (25.2)$$

Figure 25.2(a) plots the identity function. To examine the role of the bias term, note that $net_k > 0$ is equivalent to $\mathbf{w}^T \mathbf{x} > -b_k$. That is, the output transitions from negative to positive when the weighted sum of the inputs exceeds $-b_k$, as shown in Figure 25.2(b).

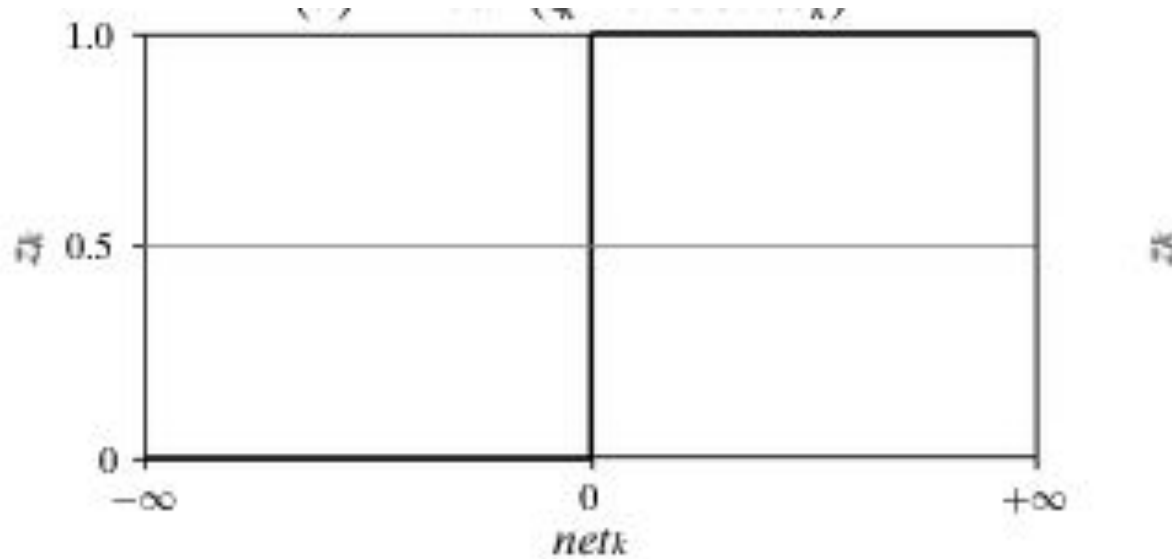
Linear Activation Function

Function: $f(\text{net}_k) = \text{net}_k$

Derivative: $\frac{\partial f(\text{net}_j)}{\partial \text{net}_j} = 1$



Step Function



(c) Step (z_k versus net_k)

Step Function: This is a binary activation function, where the neuron outputs a 0 if the net value is negative (or zero), and 1 if the net value is positive (see Figure 25.2(c)).

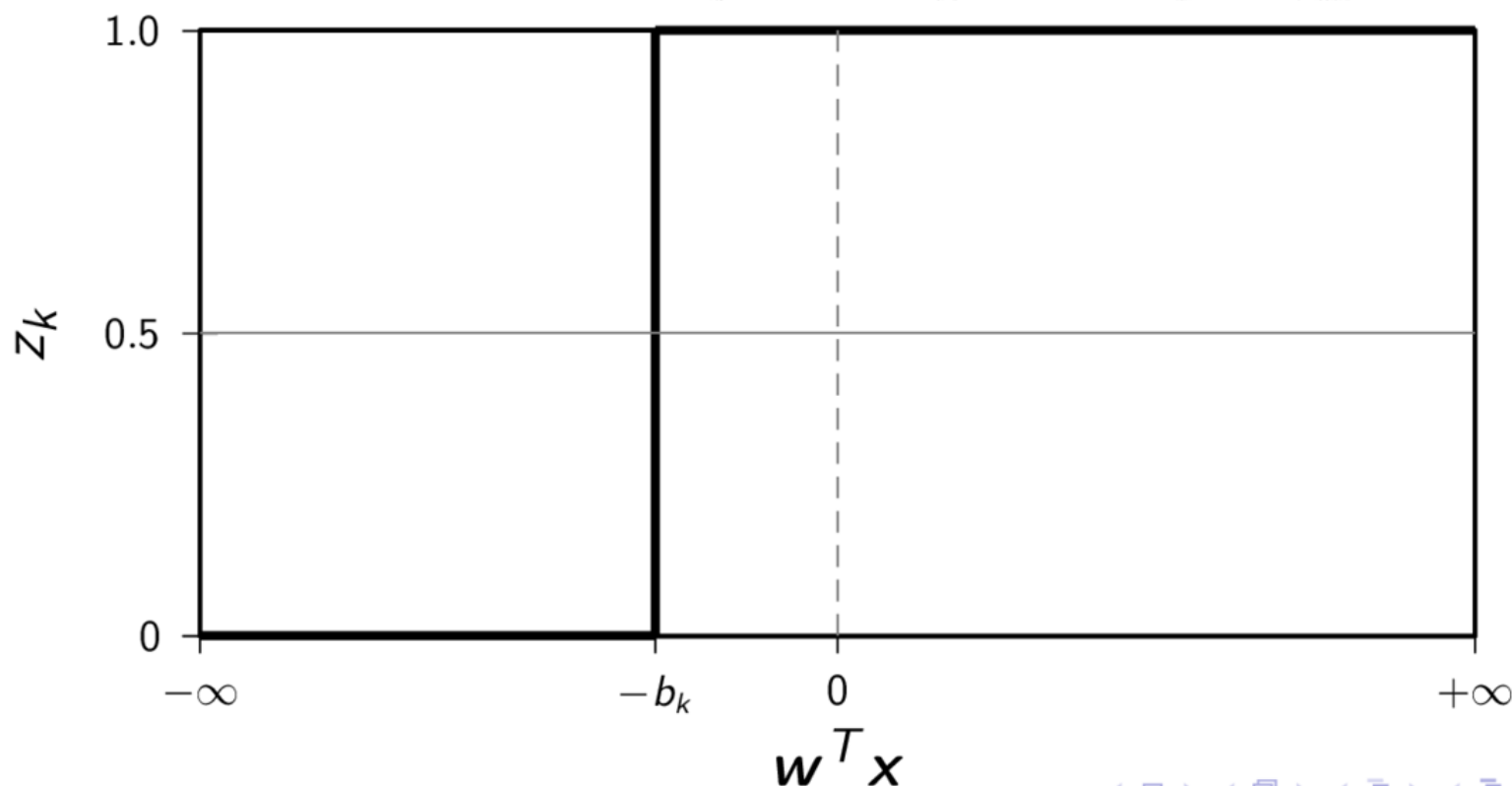
$$f(net_k) = \begin{cases} 0 & \text{if } net_k \leq 0 \\ 1 & \text{if } net_k > 0 \end{cases} \quad (25.3)$$

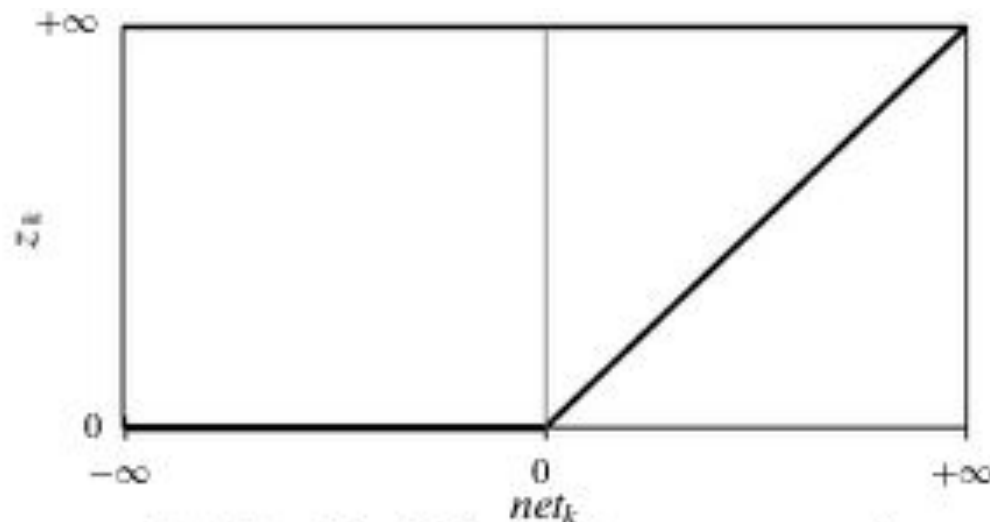
Step Activation Function

$$\text{Function: } f(\text{net}_k) = \begin{cases} 0 & \text{if } \text{net}_k \leq 0 \\ 1 & \text{if } \text{net}_k > 0 \end{cases}$$

$$\text{Derivative: } \frac{\partial f(\text{net}_j)}{\partial \text{net}_j} = 0$$

It is interesting to note that the transition from 0 to 1 happens when the weighted sum of the inputs exceeds $-b_k$, as shown in Figure 25.2(d).





(e) Rectified Linear (z_k versus net_k)

Rectified Linear Unit (ReLU): Here, the neuron remains inactive if the net input is less than or equal to zero, and then increases linearly with net_k , as shown in Figure 25.2(e).

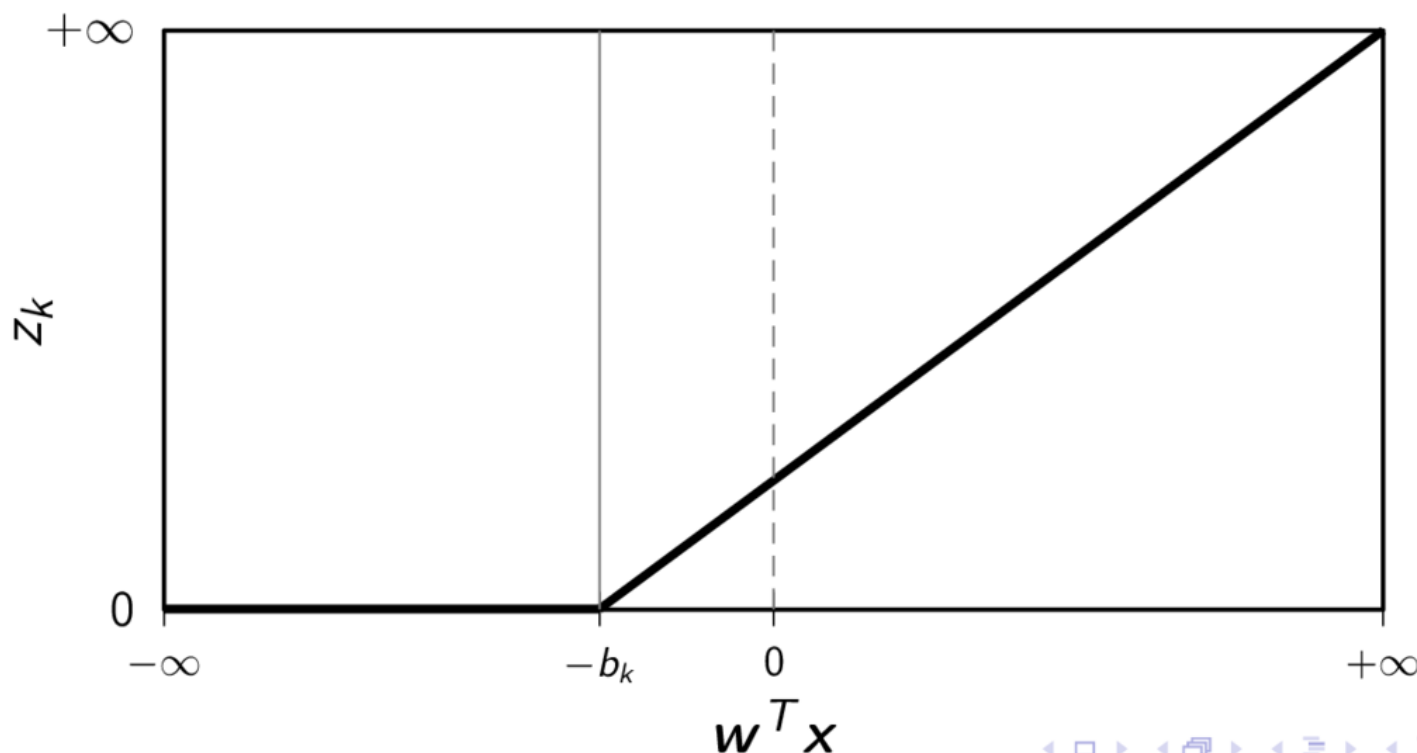
$$f(net_k) = \begin{cases} 0 & \text{if } net_k \leq 0 \\ net_k & \text{if } net_k > 0 \end{cases} \quad (25.4)$$

An alternative expression for the ReLU activation is given as $f(net_k) = \max\{0, net_k\}$. The transition from zero to linear output happens when the weighted sum of the inputs exceeds $-b_k$ (see Figure 25.2(f)).

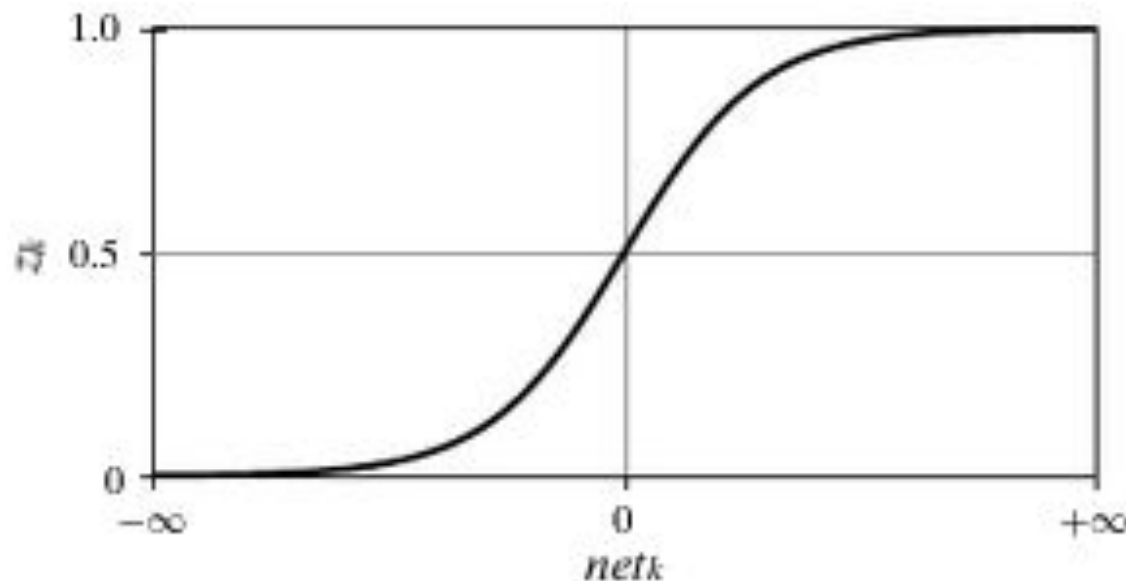
Rectified Linear Activation Function

$$\text{Function: } f(\text{net}_k) = \begin{cases} 0 & \text{if } \text{net}_k \leq 0 \\ \text{net}_k & \text{if } \text{net}_k > 0 \end{cases}$$

$$\text{Derivative: } \frac{\partial f(\text{net}_j)}{\partial \text{net}_j} = \begin{cases} 0 & \text{if } \text{net}_j \leq 0 \\ 1 & \text{if } \text{net}_j > 0 \end{cases}$$



Sigmoid Activation Function



(g) Sigmoid (z_k versus net_k)

Sigmoid: The sigmoid function, illustrated in Figure 25.2(g) squashes its input so that the output ranges between 0 and 1

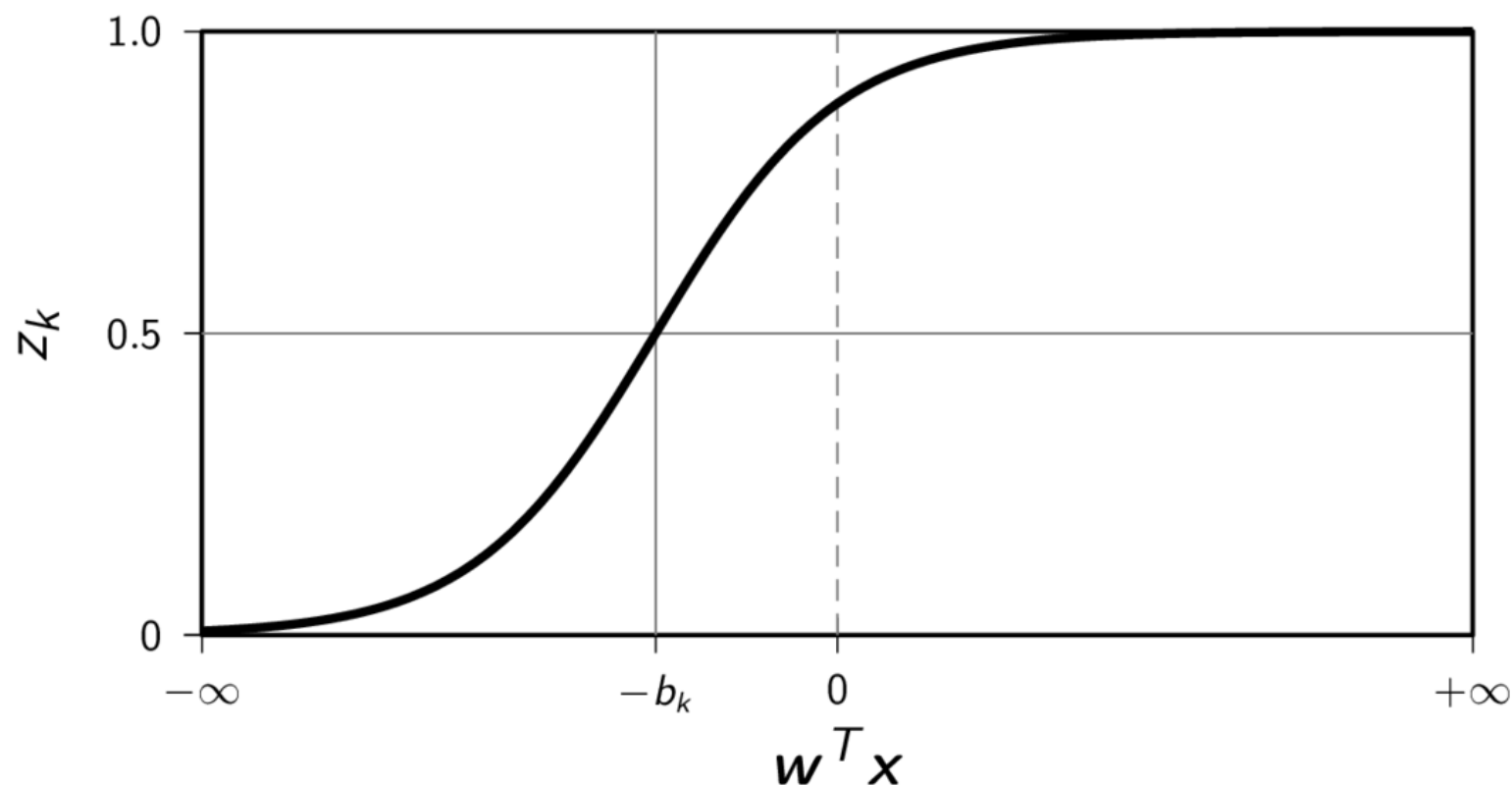
$$f(net_k) = \frac{1}{1 + \exp\{-net_k\}} \quad (25.5)$$

When $net_k = 0$, we have $f(net_k) = 0.5$, which implies that the transition point where the output crosses 0.5 happens when the weighted sum of the inputs exceeds $-b_k$ (see Figure 25.2(h)).

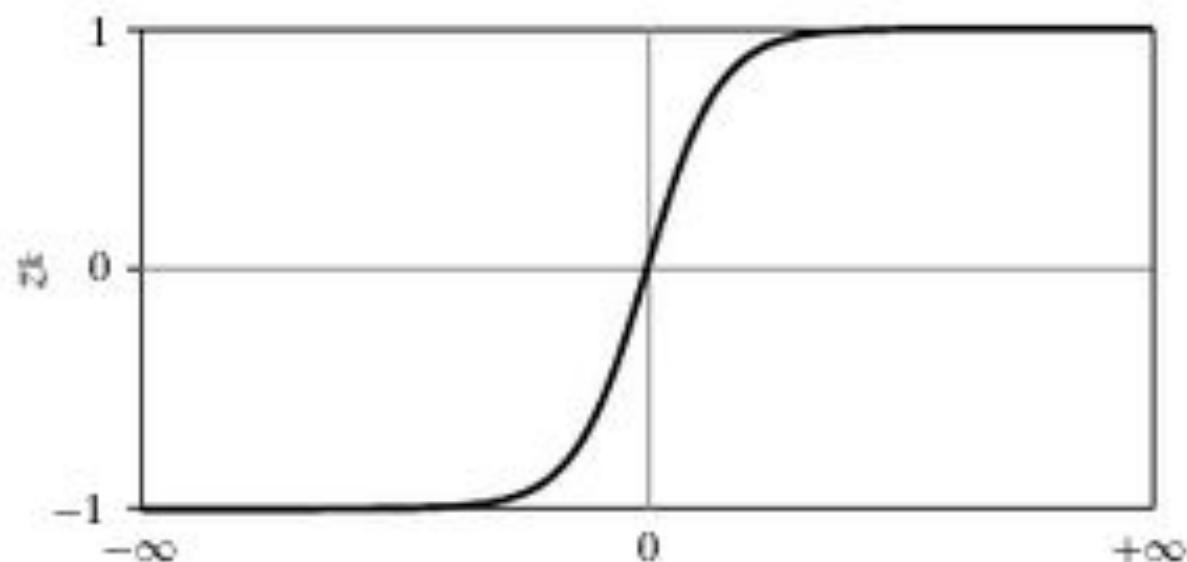
Sigmoid Activation Function

Function: $f(net_k) = \frac{1}{1 + \exp\{-net_k\}}$

Derivative: $\frac{\partial f(net_j)}{\partial net_j} = f(net_j) \cdot (1 - f(net_j))$



Hyperbolic Tangent Activation Function



Hyperbolic Tangent (tanh): The hyperbolic tangent or tanh function is similar to the sigmoid, but its output ranges between -1 and $+1$ (see Figure 25.2(i)).

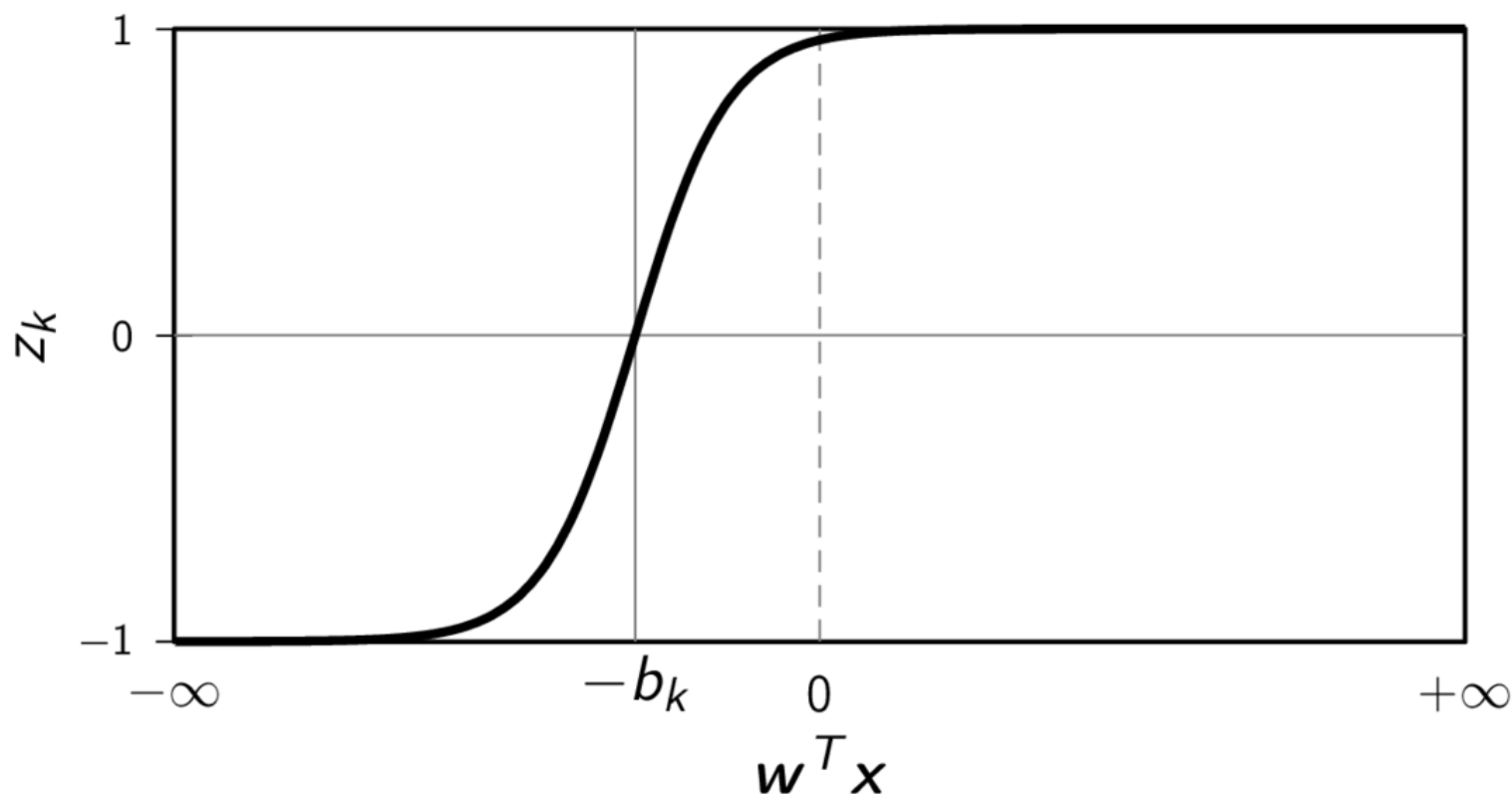
$$f(net_k) = \frac{\exp\{net_k\} - \exp\{-net_k\}}{\exp\{net_k\} + \exp\{-net_k\}} = \frac{\exp\{2 \cdot net_k\} - 1}{\exp\{2 \cdot net_k\} + 1} \quad (25.6)$$

When $net_k = 0$, we have $f(net_k) = 0$, which implies that the output transitions from negative to positive when the weighted sum of the inputs exceeds $-b_k$, as shown in Figure 25.2(j).

Hyperbolic Tangent Activation Function

Function: $f(net_k) = \frac{\exp\{net_k\} - \exp\{-net_k\}}{\exp\{net_k\} + \exp\{-net_k\}} = \frac{\exp\{2 \cdot net_k\} - 1}{\exp\{2 \cdot net_k\} + 1}$

Derivative: $\frac{\partial f(net_j)}{\partial net_j} = 1 - f^2(net_j)$

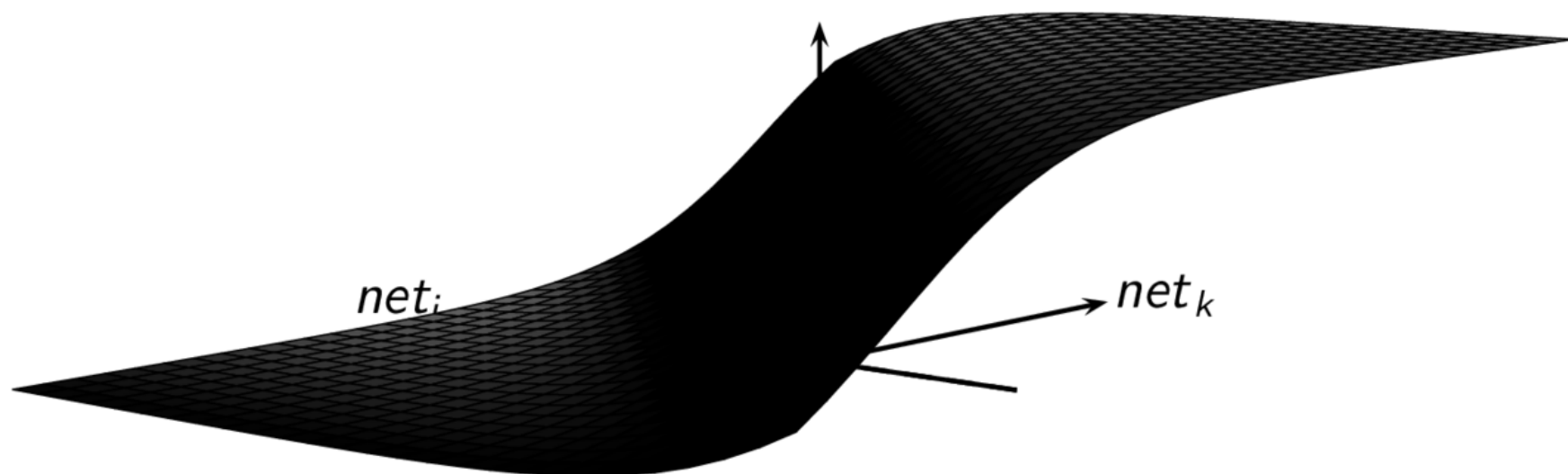


Softmax Activation Function

Function: $f(\text{net}_k | \text{net}) = \frac{\exp\{\text{net}_k\}}{\sum_{i=1}^p \exp\{\text{net}_i\}}$

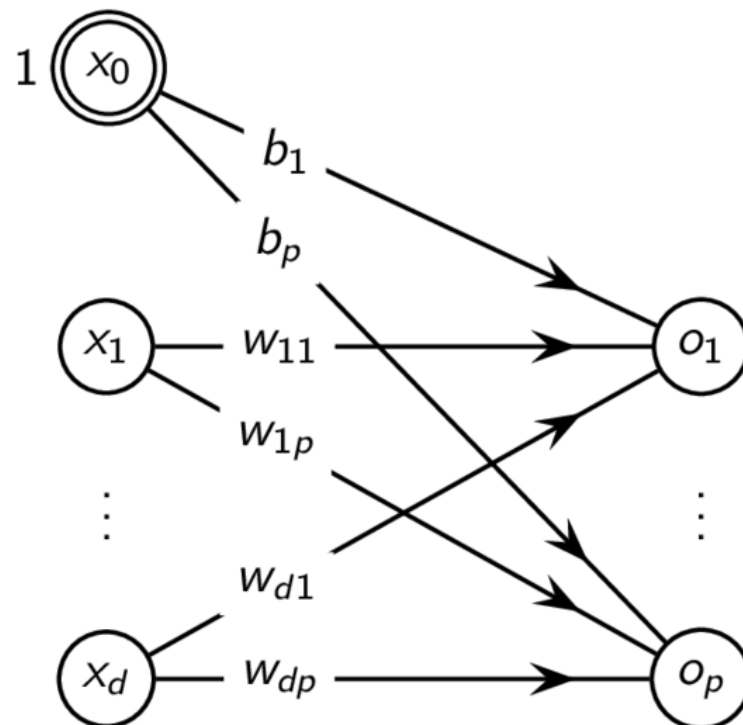
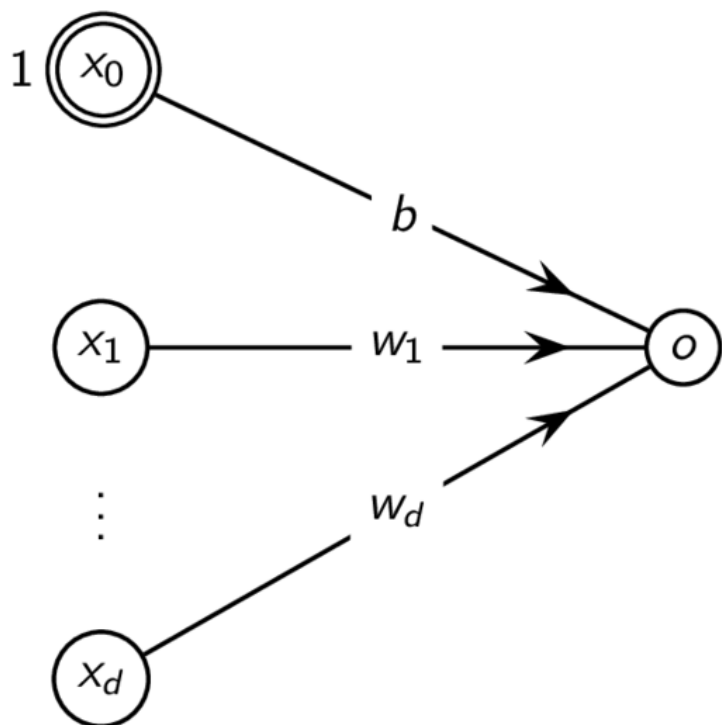
Derivative:

$$\frac{\partial f(\text{net}_j | \text{net})}{\partial \text{net}_k} = \frac{\partial o_j}{\partial \text{net}_k} = \begin{cases} o_j \cdot (1 - o_j) & \text{if } k = j \\ -o_k \cdot o_j & \text{if } k \neq j \end{cases}$$



Linear and Logistic Regression via Neural Networks

Networks of (artificial) neurons are capable of representing and learning arbitrarily complex functions for regression.



Classification via Neural Networks

Networks of artificial neurons can also learn to classify the inputs.

A simple change to the neural network allows it to solve the logistic regression problem. All we have to do is use a sigmoid activation function at the output neuron o , and use the cross-entropy error instead of squared error.

$$\mathcal{E}_x = -(y \cdot \ln(o) + (1 - y) \cdot \ln(1 - o))$$

The output of the neural network is

$$o = f(\text{net}_o) = \text{sigmoid}(b + \mathbf{w}^T \mathbf{x}) = \frac{1}{1 + \exp\{-(b + \mathbf{w}^T \mathbf{x})\}} = \pi(\mathbf{x})$$

Jai Sairam

Error Functions

Squared Error: Given an input vector $\mathbf{x} \in \mathbb{R}^d$, the squared error loss function measures the squared deviation between the predicted output vector $\mathbf{o} \in \mathbb{R}^p$ and the true response $\mathbf{y} \in \mathbb{R}^p$, defined as follows:

$$\mathcal{E}_{\mathbf{x}} = \frac{1}{2} \|\mathbf{y} - \mathbf{o}\|^2 = \frac{1}{2} \sum_{j=1}^p (y_j - o_j)^2$$

where $\mathcal{E}_{\mathbf{x}}$ denotes the error on input $\mathbf{x}$. The partial derivative of the squared error function with respect to a particular output neuron o_j is

$$\frac{\partial \mathcal{E}_{\mathbf{x}}}{\partial o_j} = \frac{1}{2} \cdot 2 \cdot (y_j - o_j) \cdot -1 = o_j - y_j$$

Across all the output neurons, we can write this as

$$\frac{\partial \mathcal{E}_{\mathbf{x}}}{\partial \mathbf{o}} = \mathbf{o} - \mathbf{y}$$

As discussed, squared error is often used in regression.

Cross-Entropy Error: For classification tasks, with K classes $\{c_1, c_2, \dots, c_K\}$, the number of output neurons $p = K$, with one output neuron per class.

Each of the classes is coded as a one-hot vector, with class c_i encoded as the i th standard basis vector $\mathbf{e}_i = (e_{i1}, e_{i2}, \dots, e_{iK})^T \in \{0, 1\}^K$, with $e_{ii} = 1$ and $e_{ij} = 0$ for all $j \neq i$.

The cross-entropy loss is defined as

$$\mathcal{E}_x = - \sum_{i=1}^K y_i \cdot \ln(o_i) = - \left(y_1 \cdot \ln(o_1) + \dots + y_K \cdot \ln(o_K) \right)$$

Note that only one element of $\mathbf{y}$ is 1 and the rest are 0.

Error Functions

The partial derivative of the cross-entropy error function with respect to a particular output neuron o_j is

$$\frac{\partial \mathcal{E}_x}{\partial o_j} = -\frac{y_j}{o_j}$$

The vector of partial derivatives of the error function with respect to the output neurons is therefore given as

$$\frac{\partial \mathcal{E}_x}{\partial \mathbf{o}} = \left(\frac{\partial \mathcal{E}_x}{\partial o_1}, \frac{\partial \mathcal{E}_x}{\partial o_2}, \dots, \frac{\partial \mathcal{E}_x}{\partial o_K} \right)^T = \left(-\frac{y_1}{o_1}, -\frac{y_2}{o_2}, \dots, -\frac{y_K}{o_K} \right)^T$$

Cross-entropy is often used for classification.

Error Functions

Binary Cross-Entropy Error: For binary classes, the positive class is 1 and the negative class is 0.

Given $\mathbf{x} \in \mathbb{R}^d$, with $y \in \{0, 1\}$, there is only one output neuron o .

$$\mathcal{E}_{\mathbf{x}} = -(y \cdot \ln(o) + (1 - y) \cdot \ln(1 - o))$$

The partial derivative with respect to the output neuron o is

$$\begin{aligned} \frac{\partial \mathcal{E}_{\mathbf{x}}}{\partial o} &= \frac{\partial}{\partial o} \left\{ -y \cdot \ln(o) - (1 - y) \cdot \ln(1 - o) \right\} \\ &= - \left(\frac{y}{o} + \frac{1 - y}{1 - o} \cdot -1 \right) = \frac{-y \cdot (1 - o) + (1 - y) \cdot o}{o \cdot (1 - o)} = \frac{o - y}{o \cdot (1 - o)} \end{aligned}$$

ANN for Multiple and Multivariate Regression

Example – Multiple Linear Regression

Consider the multiple regression of `sepal length` and `petal length` on the dependent attribute `petal width` for the Iris dataset with $n = 150$ points.

$$\hat{y} = -0.014 - 0.082 \cdot x_1 + 0.45 \cdot x_2$$

The squared error for this optimal solution is 6.179 on the training data.

A neural network, with linear activation and minimizing the SSE via gradient descent, results in

$$o = 0.0096 - 0.087 \cdot x_1 + 0.452 \cdot x_2$$

with an SSE of 6.18, which is very close to the optimal solution.

ANN for Multiple and Multivariate Regression

Example – Multivariate Linear Regression

We use the neural network architecture to learn the weights and bias for the Iris dataset, where we use `sepal length` and `sepal width` as the independent attributes, and `petal length` and `petal width` as the response or dependent attributes.

Therefore, each $\mathbf{x}_i$ is 2D, and 2-dimensional, and $\mathbf{y}_i$ is also 2D. Minimizing the SSE via gradient descent, yields:

$$\begin{pmatrix} b_1 & b_2 \\ w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} = \begin{pmatrix} -1.83 & -1.47 \\ 1.72 & 0.72 \\ -1.46 & -0.50 \end{pmatrix} \quad \begin{pmatrix} o_1 \\ o_2 \end{pmatrix} = \begin{pmatrix} -1.83 + 1.72 \cdot x_1 - 1.46 \cdot x_2 \\ -1.47 + 0.72 \cdot x_1 - 0.50 \cdot x_2 \end{pmatrix}$$

The SSE on the training set is 84.9. Optimal multivariate regression yields an SSE of 84.16:

$$\begin{pmatrix} \hat{y}_1 \\ \hat{y}_2 \end{pmatrix} = \begin{pmatrix} -2.56 + 1.78 \cdot x_1 - 1.34 \cdot x_2 \\ -1.59 + 0.73 \cdot x_1 - 0.48 \cdot x_2 \end{pmatrix}$$

Neural networks for multiclass logistic regression

Example

We applied the single output neural network with logistic activation at the output neuron and cross-entropy error function, on the Iris principal components dataset.

The output is a binary response indicating Iris-virginica ($Y = 1$) or one of the other Iris types ($Y = 0$).

As expected, the neural network learns an identical set of weights and bias as shown for the logistic regression model, namely:

$$o = -6.79 - 5.07 \cdot x_1 - 3.29 \cdot x_2$$

Next, we used a softmax activation and cross-entropy error function, to the Iris principal components data with three classes: Iris-setosa ($Y = 1$), Iris-versicolor ($Y = 2$) and Iris-virginica ($Y = 3$).

We fix the weights and bias for output neuron o_3 to be zero:

$$o_1 = -3.49 + 3.61 \cdot x_1 + 2.65 \cdot x_2$$

$$o_2 = -6.95 - 5.18 \cdot x_1 - 3.40 \cdot x_2$$

$$o_3 = 0 + 0 \cdot x_1 + 0 \cdot x_2$$

Neural networks for multiclass logistic regression

Example

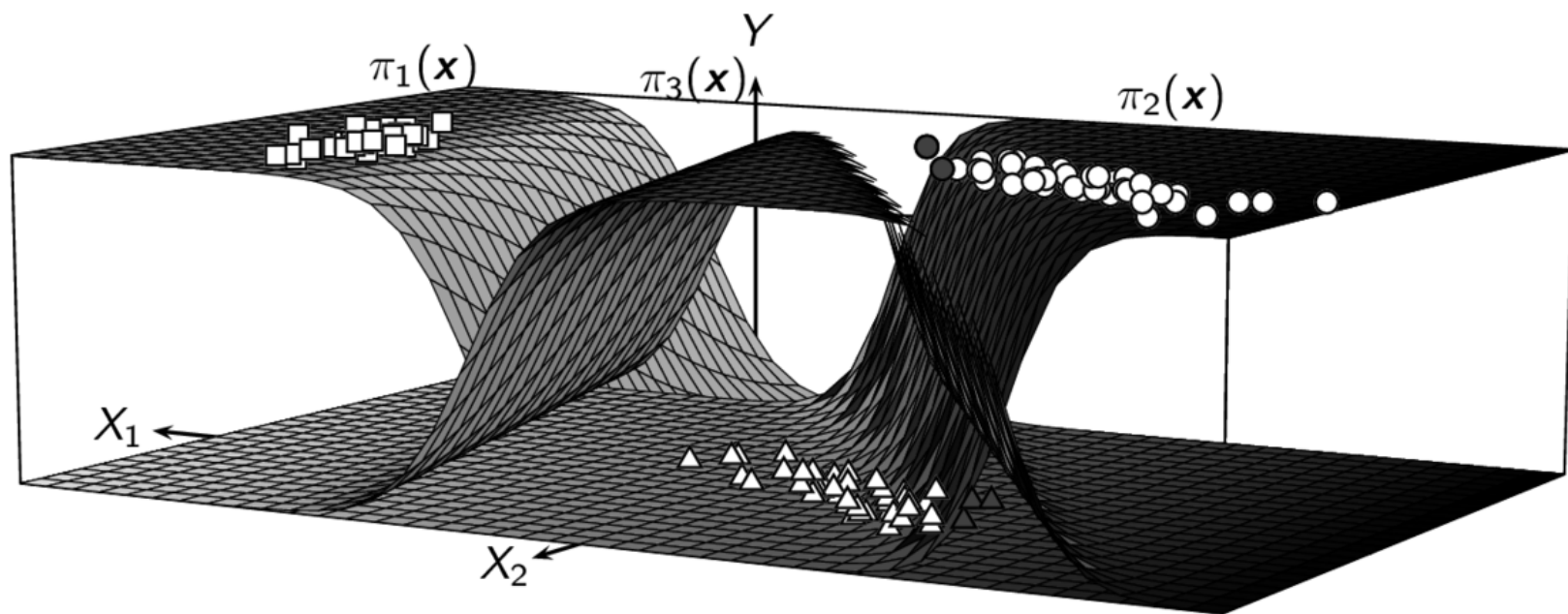
We do not constrain the weights and bias for o_3 we obtain the following model:

$$o_1 = -0.89 + 4.54 \cdot x_1 + 1.96 \cdot x_2$$

$$o_2 = -3.38 - 5.11 \cdot x_1 - 2.88 \cdot x_2$$

$$o_3 = 4.24 + 0.52 \cdot x_1 + 0.92 \cdot x_2$$

Misclassified points are shown in dark gray color. Points in class c_1 and c_2 are shown displaced with respect to the base class c_3 only for illustration.



Multilayer Perceptron: One Hidden Layer

A multilayer perceptron (MLP) is a neural network that has distinct layers of neurons.

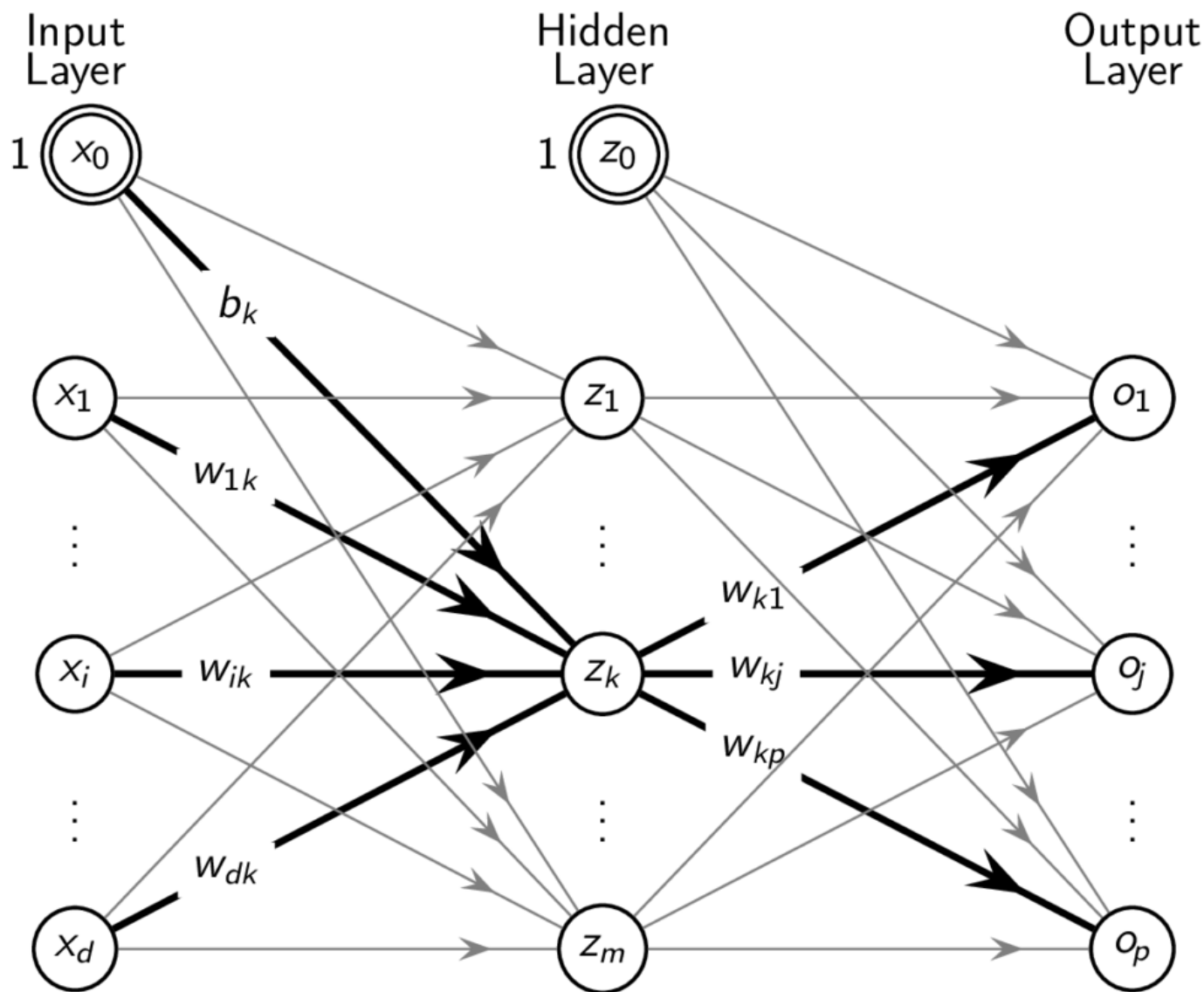
The inputs to the neural network comprise the *input layer*, and the final outputs from the MLP comprise the *output layer*.

Any intermediate layer is called a *hidden layer*, and an MLP can have one or many hidden layers.

Networks with many hidden layers are called *deep neural networks*.

An MLP is also a feed-forward network. That is, information flows in the forward direction, and from a layer only to the subsequent layer.

Multilayer Perceptron: One Hidden Layer



MLP: Feed-forward Phase

Given the input neuron values, the output for each hidden neuron z_k is:

$$z_k = f(\text{net}_k) = f\left(b_k + \sum_{i=1}^d w_{ik} \cdot x_i\right)$$

where f is some activation function, and w_{ik} denotes the weight between input neuron x_i and hidden neuron z_k .

Next, given the hidden neuron values, the value for each output neuron o_j is:

$$o_j = f(\text{net}_j) = f\left(b_j + \sum_{i=1}^m w_{ij} \cdot z_i\right)$$

where w_{ij} denotes the weight between hidden neuron z_i and output neuron o_j .

MLP: Feed-Forward Phase

The output vector can then be computed as follows:

$$\begin{aligned}\text{net}_o &= \mathbf{b}_o + \mathbf{W}_o^T \mathbf{z} \\ \mathbf{o} &= f(\text{net}_o) = f(\mathbf{b}_o + \mathbf{W}_o^T \mathbf{z})\end{aligned}$$

To summarize, for a given input $\mathbf{x} \in \mathbf{D}$ with desired response $\mathbf{y}$, an MLP computes the output vector via the feed-forward process, as follows:

$$\mathbf{o} = f(\mathbf{b}_o + \mathbf{W}_o^T \mathbf{z}) = f(\mathbf{b}_o + \mathbf{W}_o^T \cdot f(\mathbf{b}_h + \mathbf{W}_h^T \mathbf{x}))$$

where, $\mathbf{o} = (o_1, o_2, \dots, o_p)^T$ is the vector of predicted outputs from the single hidden layer MLP.

MLP: Backpropagation Phase

Backpropagation is the algorithm used to learn the weights between successive layers in an MLP.

The name comes from the manner in which the *error gradient* is propagated backwards from the output to input layers via the hidden layers.

For a given input pair $(\mathbf{x}, \mathbf{y})$ in the training data, the MLP first computes the output vector $\mathbf{o}$ via the feed-forward step.

Next, it computes the error in the predicted output *vis-a-vis* the true response $\mathbf{y}$ using the squared error function

$$\mathcal{E}_x = \frac{1}{2} \|\mathbf{y} - \mathbf{o}\|^2 = \frac{1}{2} \sum_{j=1}^p (y_j - o_j)^2$$

The basic idea is to examine the extent to which an output neuron, say o_j , deviates from the corresponding target response y_j , and to modify the weights w_{ij} between each hidden neuron z_i and o_j as some function of the error.

MLP: Backpropagation Phase

The weight update is done via a gradient descent approach to minimize the error.

Let $\nabla_{w_{ij}}$ be the gradient of the error function with respect to w_{ij} , or simply the *weight gradient* at w_{ij} .

Given the previous weight estimate w_{ij} , a new weight is computed by taking a small step η in a direction that is opposite to the weight gradient at w_{ij}

$$w_{ij} = w_{ij} - \eta \cdot \nabla_{w_{ij}}$$

In a similar manner, the bias term b_j is also updated via gradient descent

$$b_j = b_j - \eta \cdot \nabla_{b_j}$$

where ∇_{b_j} is the gradient of the error function with respect to b_j , which we call the *bias gradient* at b_j .

MLP: Backpropagation Phase

Updating Parameters Between Hidden and Output Layer

Consider the weight w_{ij} between hidden neuron z_i and output neuron o_j , and the bias term b_j between z_0 and o_j .

We compute the weight gradient at w_{ij} and bias gradient at b_j , as follows:

$$\begin{aligned}\nabla_{w_{ij}} &= \frac{\partial \mathcal{E}_x}{\partial w_{ij}} = \frac{\partial \mathcal{E}_x}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ij}} = \delta_j \cdot z_i \\ \nabla_{b_j} &= \frac{\partial \mathcal{E}_x}{\partial b_j} = \frac{\partial \mathcal{E}_x}{\partial net_j} \cdot \frac{\partial net_j}{\partial b_j} = \delta_j\end{aligned}$$

where we use the symbol δ_j to denote the partial derivative of the error with respect to net signal at o_j , which we also call the *net gradient* at o_j

Next, we compute δ_j , the net gradient at o_j .

$$\delta_j = \frac{\partial \mathcal{E}_x}{\partial net_j} = \frac{\partial \mathcal{E}_x}{\partial f(net_j)} \cdot \frac{\partial f(net_j)}{\partial net_j}$$

Note that $f(net_j) = o_j$.

MLP: Backpropagation Phase

Updating Parameters Between Hidden and Output Layer

Using the squared error function for the former, we have

$$\frac{\partial \mathcal{E}_x}{\partial f(net_j)} = \frac{\partial \mathcal{E}_x}{\partial o_j} = \frac{\partial}{\partial o_j} \left\{ \frac{1}{2} \sum_{k=1}^p (y_k - o_k)^2 \right\} = (o_j - y_j)$$

where we used the observation that all o_k for $k \neq j$ are constants with respect to o_j . Since we assume a sigmoid activation function, for the latter, we have

$$\frac{\partial f(net_j)}{\partial net_j} = o_j \cdot (1 - o_j)$$

Putting it all together, we get

$$\delta_j = (o_j - y_j) \cdot o_j \cdot (1 - o_j)$$

MLP: Backpropagation Phase

Updating Parameters Between Input and Hidden Layer

Consider w_{ij} between x_i and z_j , and b_j between x_0 and z_j .

$$\begin{aligned}\nabla_{w_{ij}} &= \frac{\partial \mathcal{E}_x}{\partial w_{ij}} = \frac{\partial \mathcal{E}_x}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ij}} = \delta_j \cdot x_i \\ \nabla_{b_j} &= \frac{\partial \mathcal{E}_x}{\partial b_j} = \frac{\partial \mathcal{E}_x}{\partial net_j} \cdot \frac{\partial net_j}{\partial b_j} = \delta_j\end{aligned}$$

δ_j at z_j has to consider the error gradients that flow back from all the output neurons to z_j .

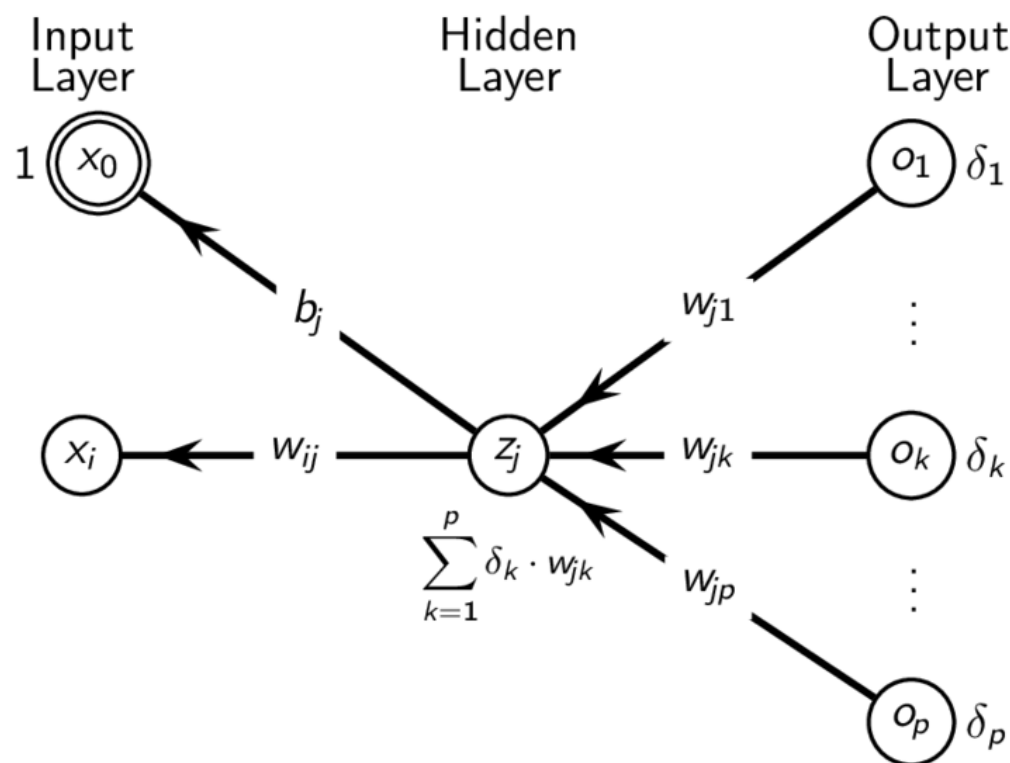
$$\begin{aligned}\delta_j &= \frac{\partial \mathcal{E}_x}{\partial net_j} = \sum_{k=1}^p \frac{\partial \mathcal{E}_x}{\partial net_k} \cdot \frac{\partial net_k}{\partial z_j} \cdot \frac{\partial z_j}{\partial net_j} = \frac{\partial z_j}{\partial net_j} \cdot \sum_{k=1}^p \frac{\partial \mathcal{E}_x}{\partial net_k} \cdot \frac{\partial net_k}{\partial z_j} \\ &= z_j \cdot (1 - z_j) \cdot \sum_{k=1}^p \delta_k \cdot w_{jk}\end{aligned}$$

where $\frac{\partial z_j}{\partial net_j} = z_j \cdot (1 - z_j)$, assuming a sigmoid activation function.

Backpropagation of gradients from output to hidden layer

To find the net gradient at z_j we have to consider the net gradients at each of the output neurons δ_k but weighted by the strength of the connection w_{jk} between z_j and o_k .

That is, we compute the weighted sum of gradients $\sum_{k=1}^p \delta_k \cdot w_{jk}$, which is used to compute δ_j , the net gradient at hidden neuron z_j .



MLP Training: Stochastic Gradient Descent

The MLP training takes multiple iterations over the input points. For each input $\mathbf{x}_i$, the MLP computes the output vector $\mathbf{o}_i$ via the feed-forward step. In the backpropagation phase, we compute the error gradient vector δ_o with respect to the net at output neurons, followed by δ_h for hidden neurons.

In the stochastic gradient descent step, we compute the error gradients with respect to the weights and biases, which are used to update the weight matrices and bias vectors.

MLP-Training ($D, m, \eta, \text{maxiter}$):

```
// Initialize bias vectors
1  $\mathbf{b}_h \leftarrow$  random  $m$ -dimensional vector with small values
2  $\mathbf{b}_o \leftarrow$  random  $p$ -dimensional vector with small values
// Initialize weight matrices
3  $\mathbf{W}_h \leftarrow$  random  $d \times m$  matrix with small values
4  $\mathbf{W}_o \leftarrow$  random  $m \times p$  matrix with small values
5  $t \leftarrow 0$  // iteration counter
```

MLP Training: Stochastic Gradient Descent

```
6 repeat
7   foreach  $(\mathbf{x}_i, \mathbf{y}_i) \in \mathbf{D}$  in random order do
8     // Feed-forward phase
9      $\mathbf{z}_i \leftarrow f(\mathbf{b}_h + \mathbf{W}_h^T \mathbf{x}_i)$ 
10     $\mathbf{o}_i \leftarrow f(\mathbf{b}_o + \mathbf{W}_o^T \mathbf{z}_i)$ 
11    // Backpropagation phase: net gradients
12     $\delta_o \leftarrow \mathbf{o}_i \odot (1 - \mathbf{o}_i) \odot (\mathbf{o}_i - \mathbf{y}_i)$ 
13     $\delta_h \leftarrow \mathbf{z}_i \odot (1 - \mathbf{z}_i) \odot (\mathbf{W}_o \cdot \delta_o)$ 
14    // Gradient descent for bias vectors
15     $\nabla_{\mathbf{b}_o} \leftarrow \delta_o; \quad \mathbf{b}_o \leftarrow \mathbf{b}_o - \eta \cdot \nabla_{\mathbf{b}_o}$ 
16     $\nabla_{\mathbf{b}_h} \leftarrow \delta_h; \quad \mathbf{b}_h \leftarrow \mathbf{b}_h - \eta \cdot \nabla_{\mathbf{b}_h}$ 
17    // Gradient descent for weight matrices
18     $\nabla_{\mathbf{W}_o} \leftarrow \mathbf{z}_i \cdot \delta_o^T; \quad \mathbf{W}_o \leftarrow \mathbf{W}_o - \eta \cdot \nabla_{\mathbf{W}_o}$ 
19     $\nabla_{\mathbf{W}_h} \leftarrow \mathbf{x}_i \cdot \delta_h^T; \quad \mathbf{W}_h \leftarrow \mathbf{W}_h - \eta \cdot \nabla_{\mathbf{W}_h}$ 
20   $t \leftarrow t + 1$ 
21 until  $t \geq \text{maxiter}$ 
```

MLP with one hidden layer

Example

Evaluate an MLP with a hidden layer using a non-linear activation function to learn the sine curve.

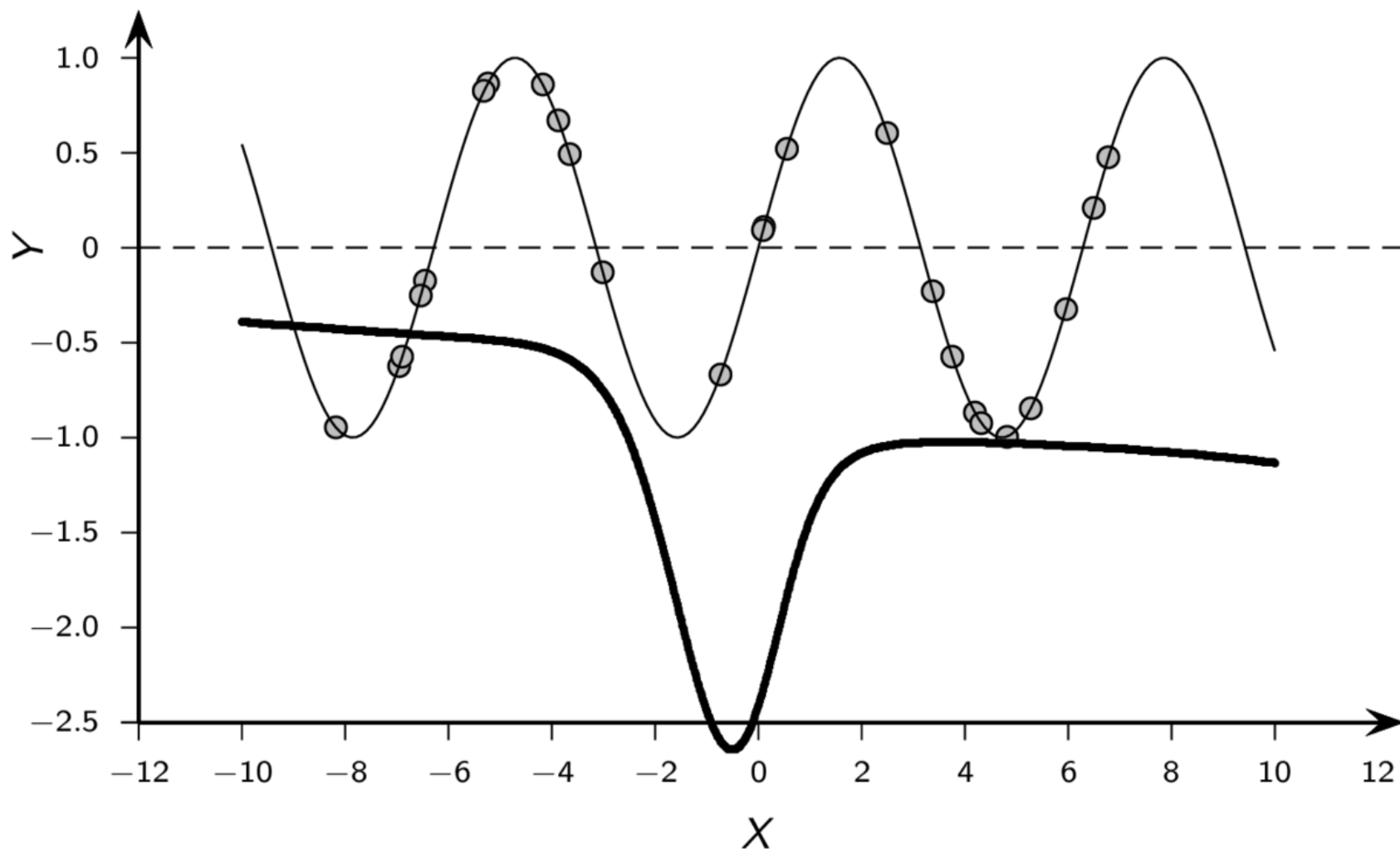
The training data comprises $n = 25$ points x_i sampled randomly in the range $[-10, 10]$, with $y_i = \sin(x_i)$.

The testing data comprises 1000 points sampled uniformly from the same range. The figure also shows the desired output curve (thin line).

We used an MLP with one input neuron ($d = 1$), ten hidden neurons ($m = 10$) and one output neuron ($p = 1$). The hidden neurons use tanh activations, whereas the output unit uses an identity activation. The step size is $\eta = 0.005$.

MLP for sine curve

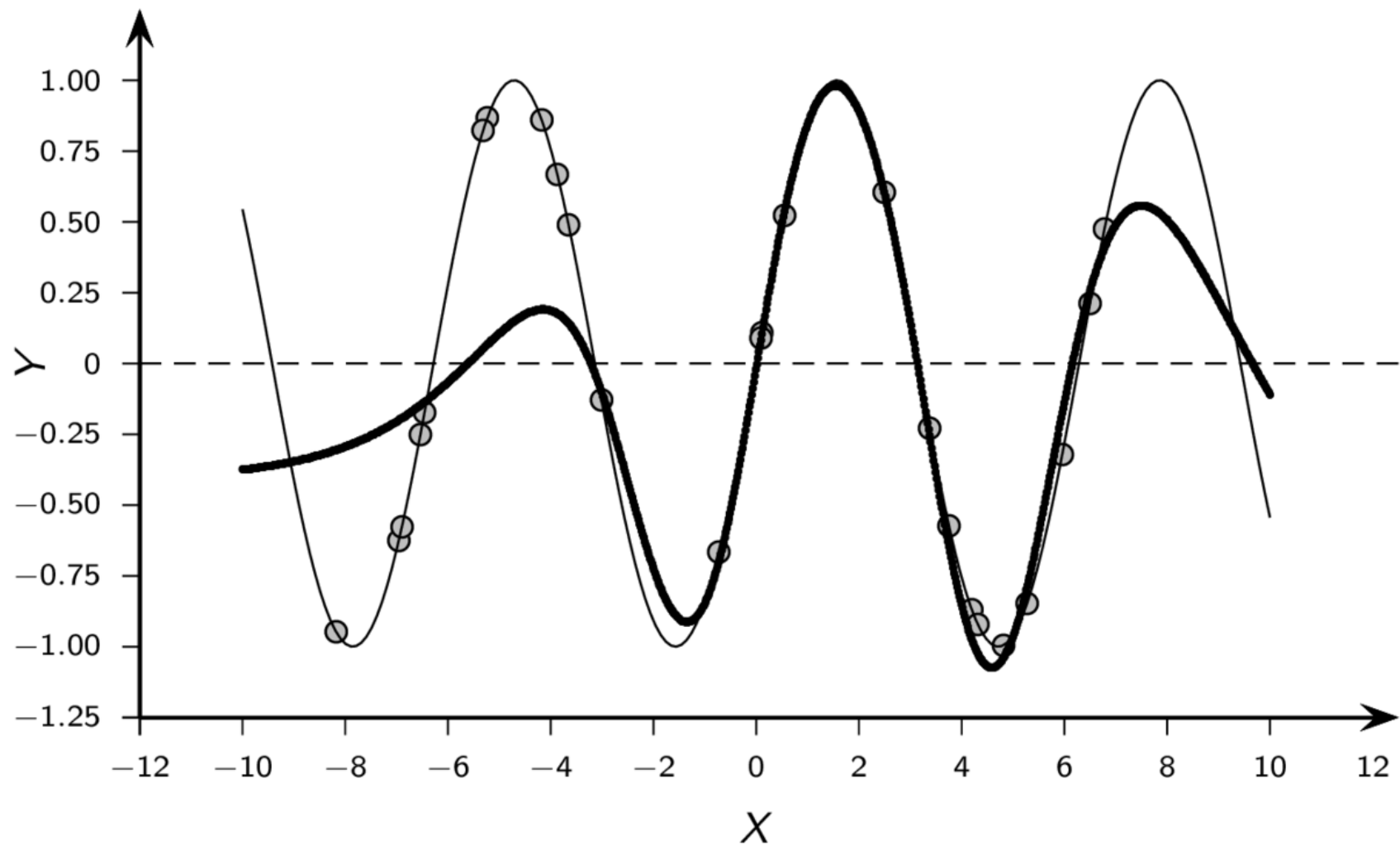
$t = 1$



$t = 1000$ 

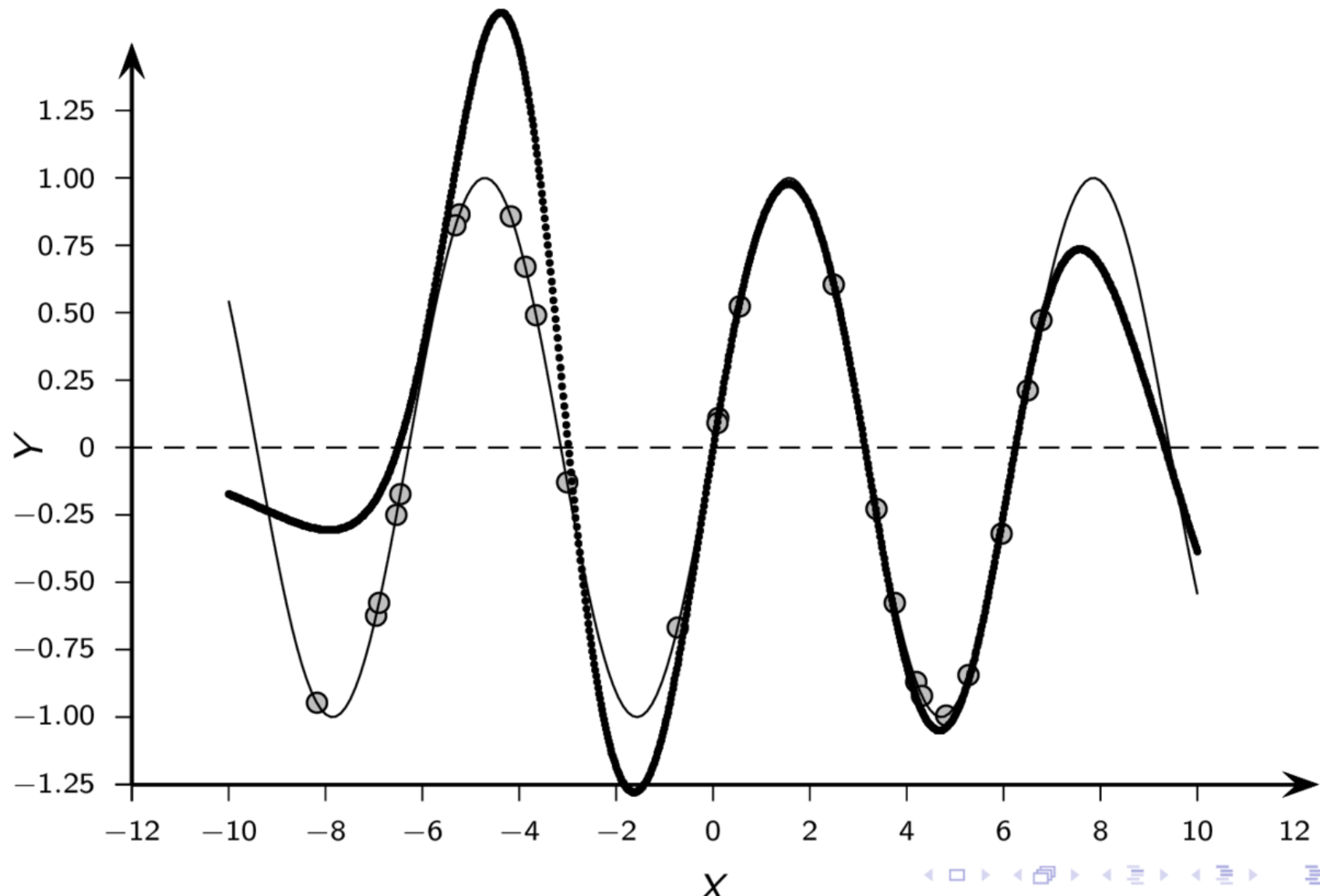
MLP for sine curve

$t = 5000$



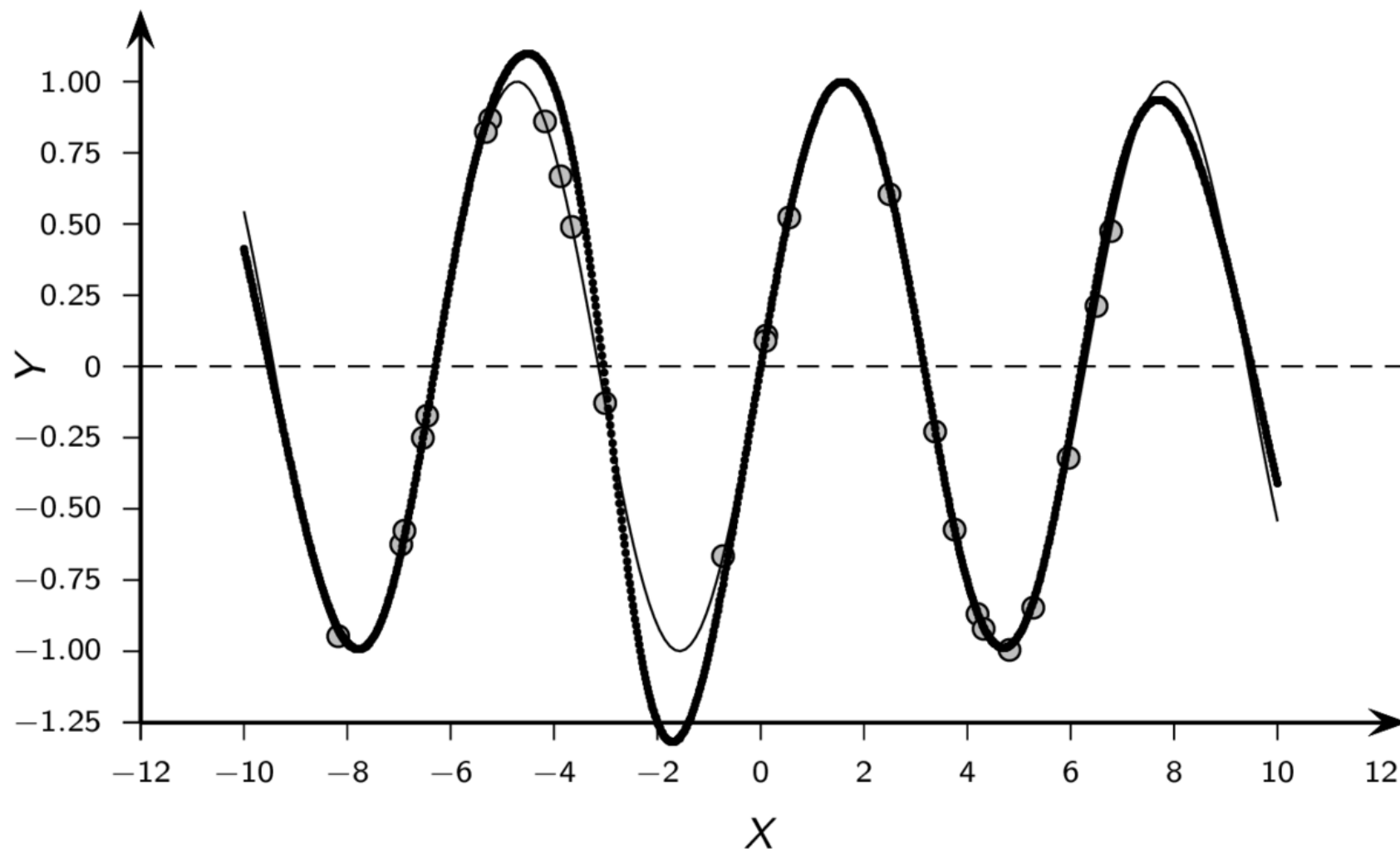
MLP for sine curve

$t = 10000$



MLP for sine curve

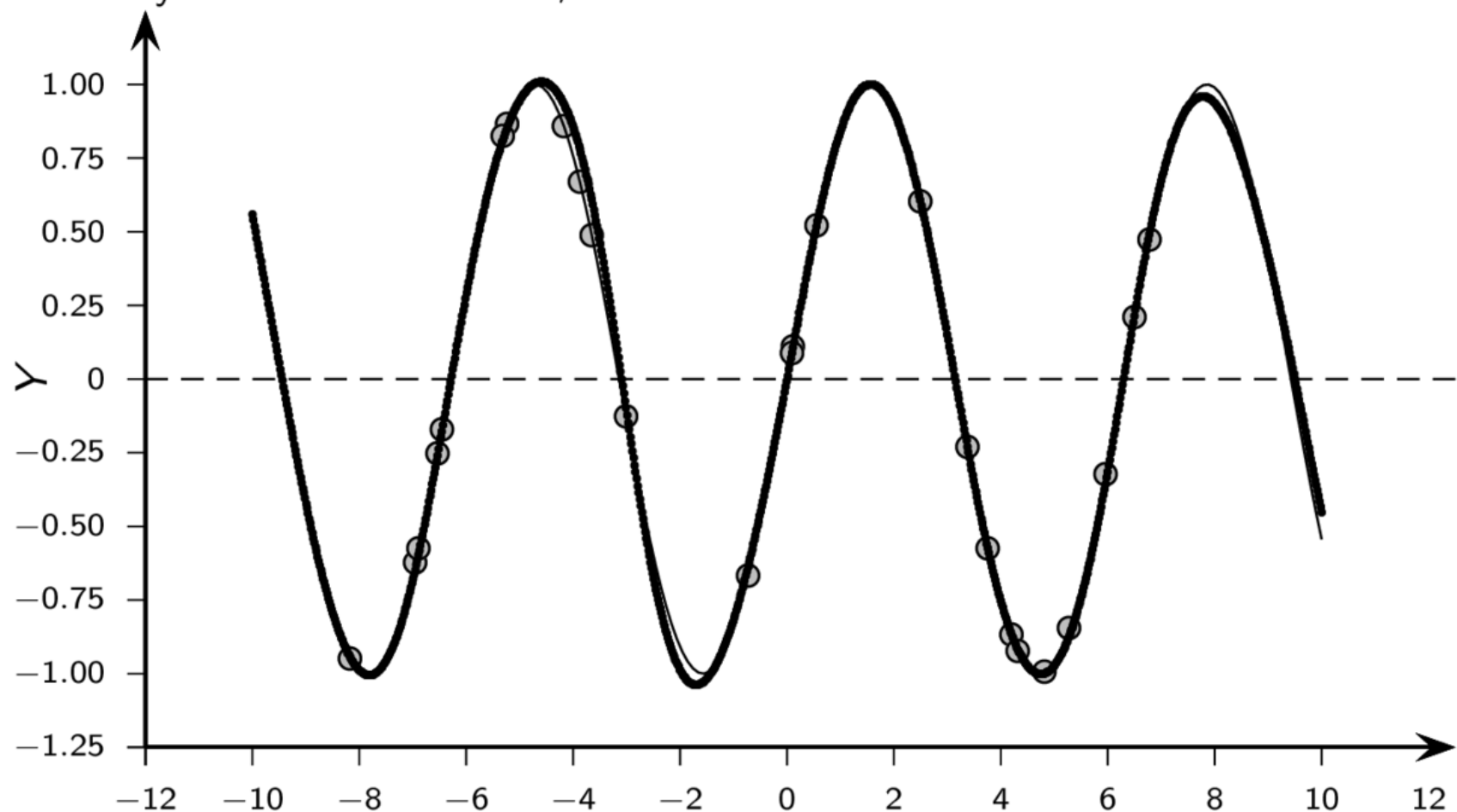
$t = 15000$



MLP for sine curve

$t = 30000$

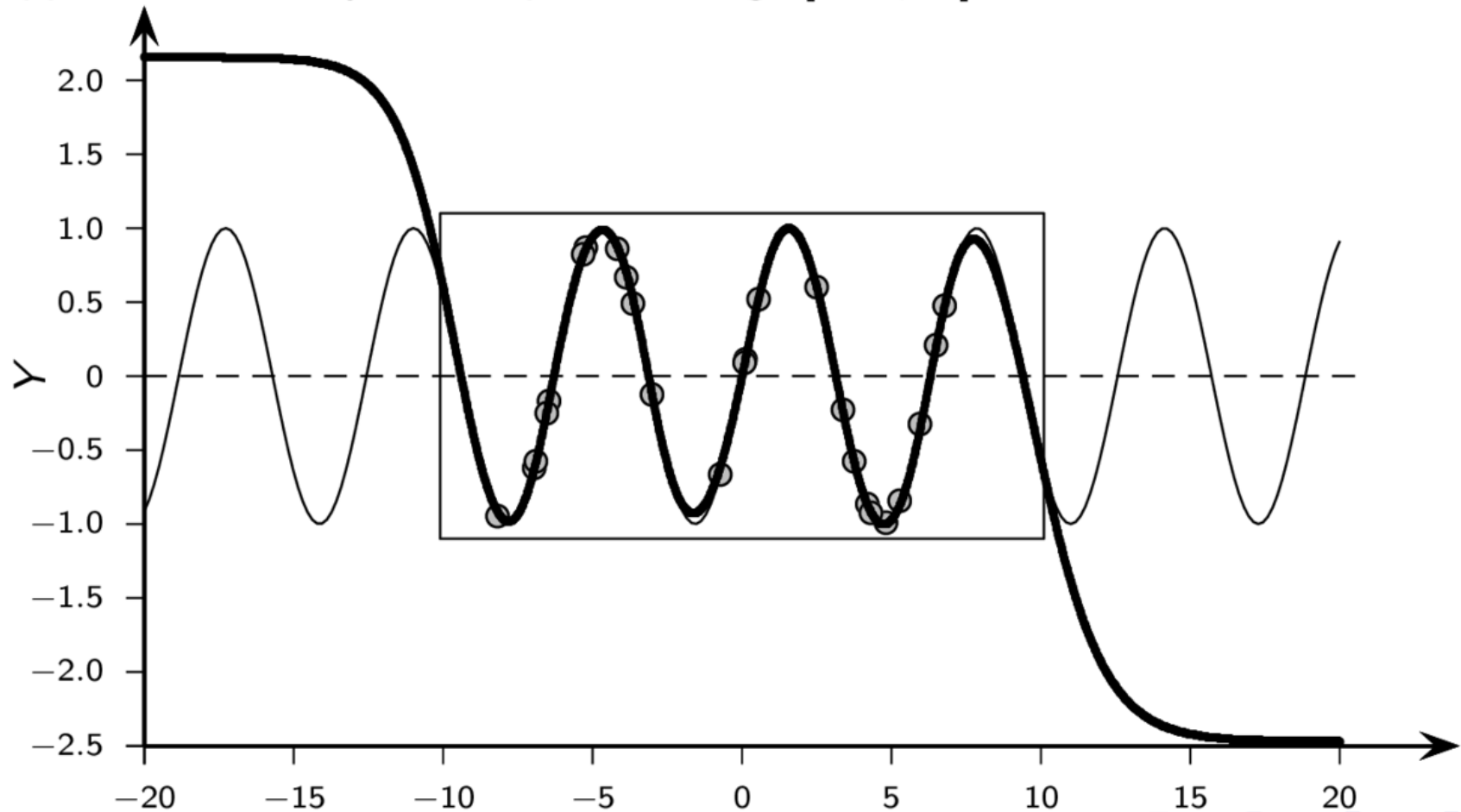
We can observe that, even with a very small training data of 25 points sampled randomly from the sine curve, the MLP is able to learn the desired function.



MLP for sine curve

Test range $[-20, 20]$

The MLP model has not really learned the sine function; rather, it has learned to approximate it only in the specified range $[-10, 10]$.

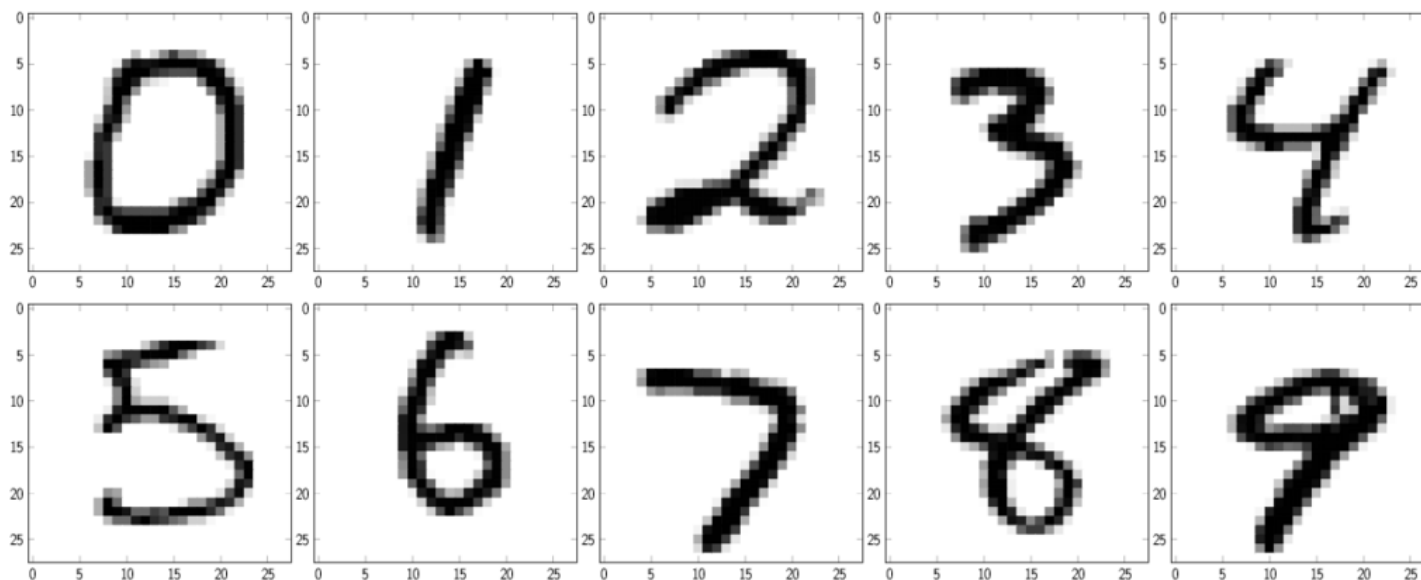


MLP for handwritten digit classification

Example

We evaluate an MLP with one hidden layer for the task of predicting the correct label for a hand-written digit from the MNIST database, which contains 60,000 training images that span the 10 digit labels, from 0 to 9.

Each (grayscale) image is a 28×28 matrix of pixels, with values between 0 and 255. Each pixel is converted to a value in the interval $[0,1]$ by dividing by 255.



MLP for handwritten digit classification

Example

Since images are 2-dimensional matrices, we first *flatten* them into a vector $\mathbf{x} \in \mathbb{R}^{784}$ with dimensionality $d = 28 \times 28 = 784$. This is done by simply concatenating all of the rows of the images to obtain one long vector.

Next, since the output labels are categorical values that denote the digits from 0 to 9, we need to convert them into binary (numerical) vectors, using *one-hot* encoding.

Thus, the label 0 is encoded as $\mathbf{e}_1 = (1, 0, 0, 0, 0, 0, 0, 0, 0, 0)^T \in \mathbb{R}^{10}$, the label 1 as $\mathbf{e}_2 = (0, 1, 0, 0, 0, 0, 0, 0, 0, 0)^T \in \mathbb{R}^{10}$, and so on, and finally the label 9 is encoded as $\mathbf{e}_{10} = (0, 0, 0, 0, 0, 0, 0, 0, 0, 1)^T \in \mathbb{R}^{10}$. That is, each input image vector $\mathbf{x}$ has a corresponding target response vector $\mathbf{y} \in \{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_{10}\}$.

Thus, the input layer for the MLP has $d = 784$ neurons, and the output layer has $p = 10$ neurons.

MLP for handwritten digit classification

Example

For the hidden layer, we consider several MLP models, each with a different number of hidden neurons m . We try $m = 0, 7, 49, 98, 196, 392$, to study the effect of increasing the number of hidden neurons, from small to large.

For the hidden layer, we use ReLU activation function, and for the output layer, we use softmax activation, since the target response vector has only one neuron with value 1, with the rest being 0.

Note that $m = 0$ means that there is no hidden layer – the input layer is directly connected to the output layer, which is equivalent to a multiclass logistic regression model. We train each MLP for $t = 15$ epochs, using step size $\eta = 0.25$.

MLP for handwritten digit classification

Example

The final test error at the end of training is given as

m	0	7	10	49	98	196	392
errors	1677	901	792	546	495	470	454

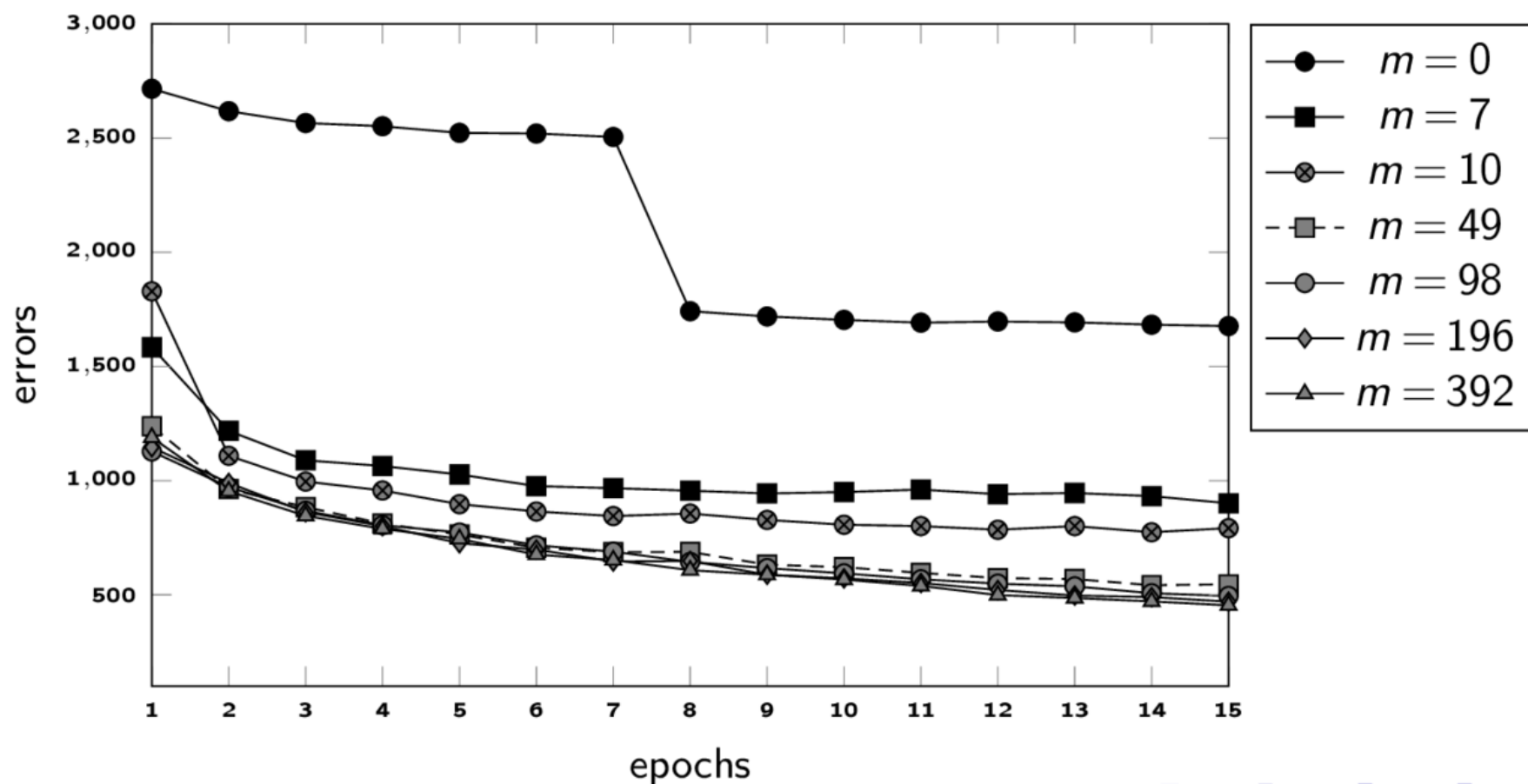
We can observe that adding a hidden layer significantly improves the prediction accuracy. Using even a small number of hidden neurons helps, compared to the logistic regression model ($m = 0$). For example, using $m = 7$ results in 901 errors (or error rate 9.01%) compared to using $m = 0$, which results in 1677 errors (or error rate 16.77%).

On the other hand, as we increase the number of hidden neurons, the error rate decreases, though with diminishing returns. Using $m = 196$, the error rate is 4.70%, but even after doubling the number of hidden neurons ($m = 392$), the error rate goes down to only 4.54%. Further increasing m does not reduce the error rate.

MNIST: Prediction error as a function of epochs

During training, we plot the number of misclassified images after each epoch, on the separate MNIST test set comprising 10,000 images.

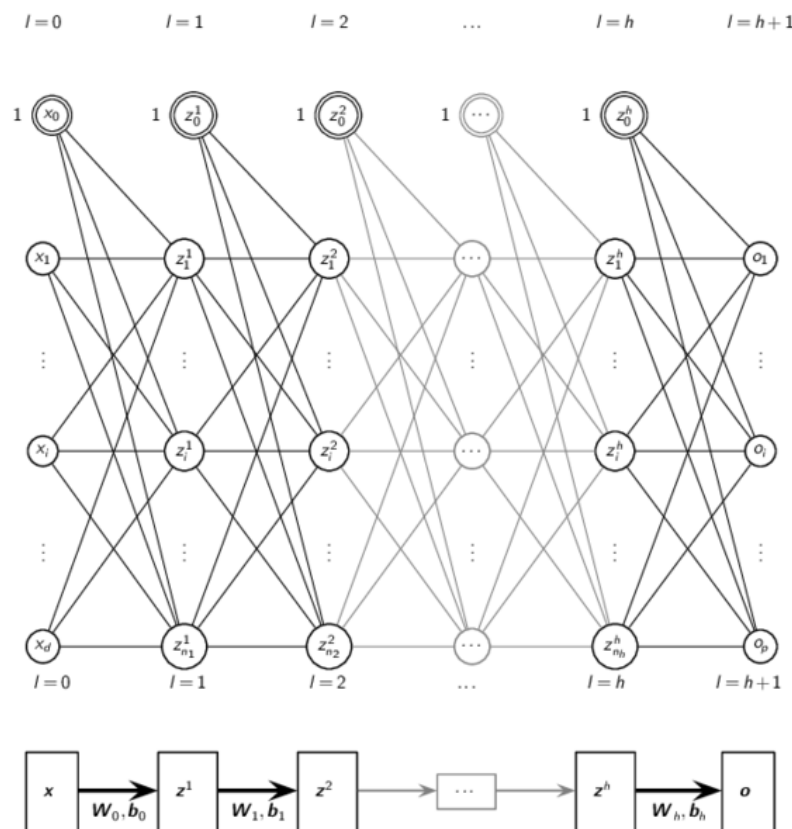
Figure shows the number of errors from each of the models (with a different number of hidden neurons m), after each epoch.



Deep Multilayer Perceptron

We now generalize the feed-forward and backpropagation steps for many hidden layers, as well as arbitrary error and neuron activation functions.

Consider an MLP with h hidden layers, n input points $\mathbf{x}_i \in \mathbb{R}^d$ ($l = 0$), and the response vector $\mathbf{y}_i \in \mathbb{R}^p$ ($l = h + 1$).



Deep Multilayer Perceptron

Feed-forward Phase

Typically in a deep MLP, the same activation function f^l is used for all neurons in a given layer l .

The input layer always uses the identity activation, so f^0 is the identity function. Also, all bias neurons also use the identity function with a fixed value of 1.

The hidden layers typically use sigmoid, tanh, or ReLU activations.

The output layer typically uses sigmoid or softmax activations for classification tasks, or identity activations for regression tasks.

For $(\mathbf{x}, \mathbf{y}) \in \mathbf{D}$, the deep MLP computes the output vector as:

$$\begin{aligned} \mathbf{o} &= f^{h+1}(\mathbf{b}_h + \mathbf{W}_h^T \cdot \mathbf{z}^h) \\ &= f^{h+1}(\mathbf{b}_h + \mathbf{W}_h^T \cdot f^h(\mathbf{b}_{h-1} + \mathbf{W}_{h-1}^T \cdot \mathbf{z}^{h-1})) \\ &= \vdots \\ &= f^{h+1}\left(\mathbf{b}_h + \mathbf{W}_h^T \cdot f^h\left(\mathbf{b}_{h-1} + \mathbf{W}_{h-1}^T \cdot f^{h-1}\left(\dots f^2\left(\mathbf{b}_1 + \mathbf{W}_1^T \cdot f^1(\mathbf{b}_0 + \mathbf{W}_0^T \cdot \mathbf{x})\right)\right)\right)\right) \end{aligned}$$

Deep Multilayer Perceptron

Backpropagation Phase

Consider the weight update between a given layer and another, including between the input and hidden layer, or between two hidden layers, or between the last hidden layer and the output layer.

Let z_i^l be a neuron in layer l , and z_j^{l+1} a neuron in the next layer $l+1$. Let w_{ij}^l be the weight between z_i^l and z_j^{l+1} , and let b_j^l denote the bias term between z_0^l and z_j^{l+1} .

The weight and bias are updated using the gradient descent approach

$$w_{ij}^l = w_{ij}^l - \eta \cdot \nabla_{w_{ij}^l} \quad b_j^l = b_j^l - \eta \cdot \nabla_{b_j^l}$$

where $\nabla_{w_{ij}^l}$ is the weight gradient and $\nabla_{b_j^l}$ is the bias gradient, i.e., the partial derivative of the error function with respect to the weight and bias, respectively.

Deep Multilayer Perceptron

Backpropagation Phase

We can use the chain rule to write the weight and bias gradient, as follows

$$\nabla_{w_{ij}^l} = \frac{\partial \mathcal{E}_x}{\partial w_{ij}^l} = \frac{\partial \mathcal{E}_x}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ij}^l} = \delta_j^{l+1} \cdot z_i^l = z_i^l \cdot \delta_j^{l+1}$$
$$\nabla_{b_j^l} = \frac{\partial \mathcal{E}_x}{\partial b_j^l} = \frac{\partial \mathcal{E}_x}{\partial net_j} \cdot \frac{\partial net_j}{\partial b_j^l} = \delta_j^{l+1}$$

In summary, the update of the weights and biases is

$$\begin{aligned} \mathbf{W}_l &= \mathbf{W}_l - \eta \cdot \nabla_{\mathbf{W}_l} \\ \mathbf{b}_l &= \mathbf{b}_l - \eta \cdot \nabla_{\mathbf{b}_l} \end{aligned}$$

where η is the step size. However, we observe that to compute the weight and bias gradients for layer l we need to compute the net gradients δ^{l+1} at layer $l + 1$.

Deep Multilayer Perceptron

Net Gradients at Output Layer

If all of the output neurons are independent (for example, when using linear or sigmoid activations), the net gradient is obtained by differentiating the error function with respect to the net signal at the output neurons. That is,

$$\delta_j^{h+1} = \frac{\partial \mathcal{E}_x}{\partial \text{net}_j} = \frac{\partial \mathcal{E}_x}{\partial f^{h+1}(\text{net}_j)} \cdot \frac{\partial f^{h+1}(\text{net}_j)}{\partial \text{net}_j} = \frac{\partial \mathcal{E}_x}{\partial o_j} \cdot \frac{\partial f^{h+1}(\text{net}_j)}{\partial \text{net}_j}$$

If the output neurons are not independent (for example, when using a softmax activation), then :

$$\delta_j^{h+1} = \frac{\partial \mathcal{E}_x}{\partial \text{net}_j} = \sum_{i=1}^p \frac{\partial \mathcal{E}_x}{\partial f^{h+1}(\text{net}_i)} \cdot \frac{\partial f^{h+1}(\text{net}_i)}{\partial \text{net}_j}$$

For regression, we use the SSE with linear activation function, whereas for logistic regression and classification, we use the cross-entropy error function with a sigmoid activation for binary classes, and softmax activation for multiclass problems.

Deep Multilayer Perceptron

Net Gradients at Hidden Layers

Let us assume that we have already computed the net gradients at layer $l + 1$, namely δ^{l+1} .

Since neuron z_j^l in layer l is connected to all of the neurons in layer $l + 1$ (except for the bias neuron z_0^{l+1}), to compute the net gradient at z_j^l , we have to account for the error from each neuron in layer $l + 1$, as follows:

$$\begin{aligned}\delta_j^l &= \frac{\partial \mathcal{E}_x}{\partial net_j} = \sum_{k=1}^{n_{l+1}} \frac{\partial \mathcal{E}_x}{\partial net_k} \cdot \frac{\partial net_k}{\partial f^l(net_j)} \cdot \frac{\partial f^l(net_j)}{\partial net_j} \\ &= \frac{\partial f^l(net_j)}{\partial net_j} \cdot \sum_{k=1}^{n_{l+1}} \delta_k^{l+1} \cdot w_{jk}^l\end{aligned}$$

So the net gradient at z_j^l in layer l depends on the derivative of the activation function with respect to its net_j , and the weighted sum of the net gradients from all the neurons z_k^{l+1} at the next layer $l + 1$.

Net Gradients at Hidden Layers

For the commonly used activation functions at the hidden layer, we have

$$\partial \mathbf{f}^l = \begin{cases} 1 & \text{for linear} \\ \mathbf{z}^l(1 - \mathbf{z}^l) & \text{for sigmoid} \\ (1 - \mathbf{z}^l \odot \mathbf{z}^l) & \text{for tanh} \end{cases}$$

The net gradients are computed recursively, starting from the output layer $h + 1$, then hidden layer h , and so on, until we finally compute the net gradients at the first hidden layer $l = 1$. That is,

$$\begin{aligned} \delta^h &= \partial \mathbf{f}^h \odot (\mathbf{W}_h \cdot \delta^{h+1}) \\ \delta^{h-1} &= \partial \mathbf{f}^{h-1} \odot (\mathbf{W}_{h-1} \cdot \delta^h) = \partial \mathbf{f}^{h-1} \odot \left(\mathbf{W}_{h-1} \cdot (\partial \mathbf{f}^h \odot (\mathbf{W}_h \cdot \delta^{h+1})) \right) \\ &\vdots \\ \delta^1 &= \partial \mathbf{f}^1 \odot \left(\mathbf{W}_1 \cdot \left(\partial \mathbf{f}^2 \odot \left(\mathbf{W}_2 \cdots (\partial \mathbf{f}^h \odot (\mathbf{W}_h \cdot \delta^{h+1})) \right) \right) \right) \end{aligned}$$

Deep MLP Training: Stochastic Gradient Descent

Deep-MLP-Training ($D, h, \eta, \text{maxiter}, n_1, n_2, \dots, n_h, f^1, f^2, \dots, f^{h+1}$):

- 1 $n_0 \leftarrow d$ // input layer size
- 2 $n_{h+1} \leftarrow p$ // output layer size
// Initialize weight matrices and bias vectors
- 3 **for** $l = 0, 1, 2, \dots, h$ **do**
- 4 $\mathbf{b}_l \leftarrow$ random n_{l+1} vector with small values
- 5 $\mathbf{W}_l \leftarrow$ random $n_l \times n_{l+1}$ matrix with small values
- 6 $t \leftarrow 0$ // iteration counter

Deep MLP Training: Stochastic Gradient Descent

```
7 repeat
8   foreach  $(x_i, y_i) \in D$  in random order do
9        $z^0 \leftarrow x_i$  // Feed-Forward Phase
10      for  $l = 0, 1, 2, \dots, h$  do  $z^{l+1} \leftarrow f^{l+1}(b_l + W_l^T \cdot z^l)$ 
11       $o_i \leftarrow z^{h+1}$ 
12      if independent outputs then // Backpropagation phase
13           $\delta^{h+1} \leftarrow \partial f^{h+1} \odot \partial \mathcal{E}_{x_i}$  // net gradients at output
14      else
15           $\delta^{h+1} \leftarrow \partial F^{h+1} \cdot \partial \mathcal{E}_{x_i}$  // net gradients at output
16      for  $l = h, h-1, \dots, 1$  do  $\delta^l \leftarrow \partial f^l \odot (W_l \cdot \delta^{l+1})$  // net gradients
17      for  $l = 0, 1, \dots, h$  do // Gradient Descent Step
18           $\nabla_{W_l} \leftarrow z^l \cdot (\delta^{l+1})^T$  // weight gradient matrix at layer  $l$ 
19           $\nabla_{b_l} \leftarrow \delta^{l+1}$  // bias gradient vector at layer  $l$ 
20      for  $l = 0, 1, \dots, h$  do
21           $W_l \leftarrow W_l - \eta \cdot \nabla_{W_l}$  // update  $W_l$ 
22           $b_l \leftarrow b_l - \eta \cdot \nabla_{b_l}$  // update  $b_l$ 
23       $t \leftarrow t + 1$ 
24 until  $t \geq \text{maxiter}$ 
```

Vanishing or Exploding Gradients

In the *vanishing gradient* problem, the norm of the net gradient can decay exponentially with the distance from the output layer, that is, as we backpropagate the gradients from the output layer to the input layer. In this case the network will learn extremely slowly, if at all, since the gradient descent method will make minuscule changes to the weights and biases.

On the other hand, in the *exploding gradient* problem, the norm of the net gradient can grow exponentially with the distance from the output layer. In this case, the weights and biases will become exponentially large, resulting in a failure to learn. The gradient explosion problem can be mitigated to some extent by *gradient thresholding*, that is, by resetting the value if it exceeds an upper bound.

The vanishing gradients problem is more difficult to address. Typically sigmoid activations are more susceptible to this problem, and one solution is to use alternative activation functions such as ReLU. In general, recurrent neural networks, which are deep neural networks with *feedback* connections, are more prone to vanishing and exploding gradients.

Deep MLP

Example

We now examine deep MLPs for predicting the labels for the MNIST handwritten images dataset.

Recall that this dataset has $n = 60000$ grayscale images of size 28×28 that we treat as $d = 784$ dimensional vectors. The pixel values between 0 and 255 are converted to the range 0 and 1 by dividing each value by 255. The target response vector is a one-hot encoded vector for class labels $\{0, 1, \dots, 9\}$.

Thus, the input to the MLP $\mathbf{x}_i$ has dimensionality $d = 784$, and the output layer has dimensionality $p = 10$. We use softmax activation for the output layer. We use ReLU activation for the hidden layers, and consider several deep models with different number and sizes of the hidden layers. We use step size $\eta = 0.3$ and train for $t = 15$ epochs. Training was done using minibatches, using batch size of 1000.

Deep MLP

Example

We evaluate the performance of each MLP on the MNIST test dataset that contains 10,000 images. The final test error at the end of training is given as

hidden layers	errors
$n_1 = 392$	396
$n_1 = 196, n_2 = 49$	303
$n_1 = 392, n_2 = 196, n_3 = 49$	290
$n_1 = 392, n_2 = 196, n_3 = 98, n_4 = 49$	278

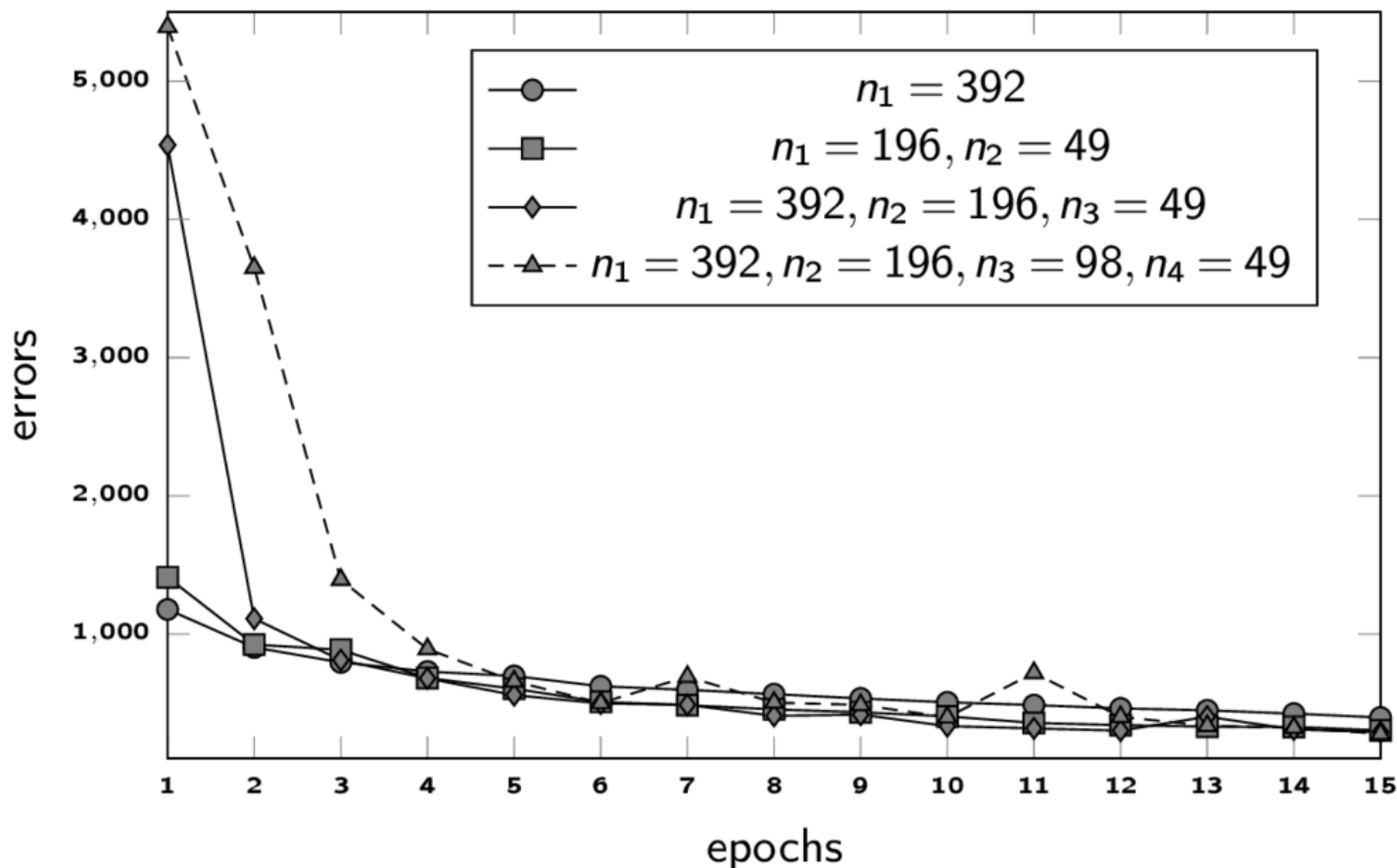
We can observe that as we increase the number of layers, we do get performance improvements.

The deep MLP with four hidden layers of sizes $n_1 = 392, n_2 = 196, n_3 = 98, n_4 = 49$ results in an error rate of 2.78% on the training set, whereas the MLP with a single hidden layer of size $n_1 = 392$ has an error rate of 3.96%.

MNIST: Deep MLPs

Prediction error as a function of epochs.

The deeper MLP improves the prediction accuracy, but adding more layers does not reduce the error rate, and can also lead to performance degradation.



Data Mining and Machine Learning: Fundamental Concepts and Algorithms

dataminingbook.info

Mohammed J. Zaki¹ Wagner Meira Jr.²

¹Department of Computer Science
Rensselaer Polytechnic Institute, Troy, NY, USA

²Department of Computer Science
Universidade Federal de Minas Gerais, Belo Horizonte, Brazil

Chapter 25: Neural Networks